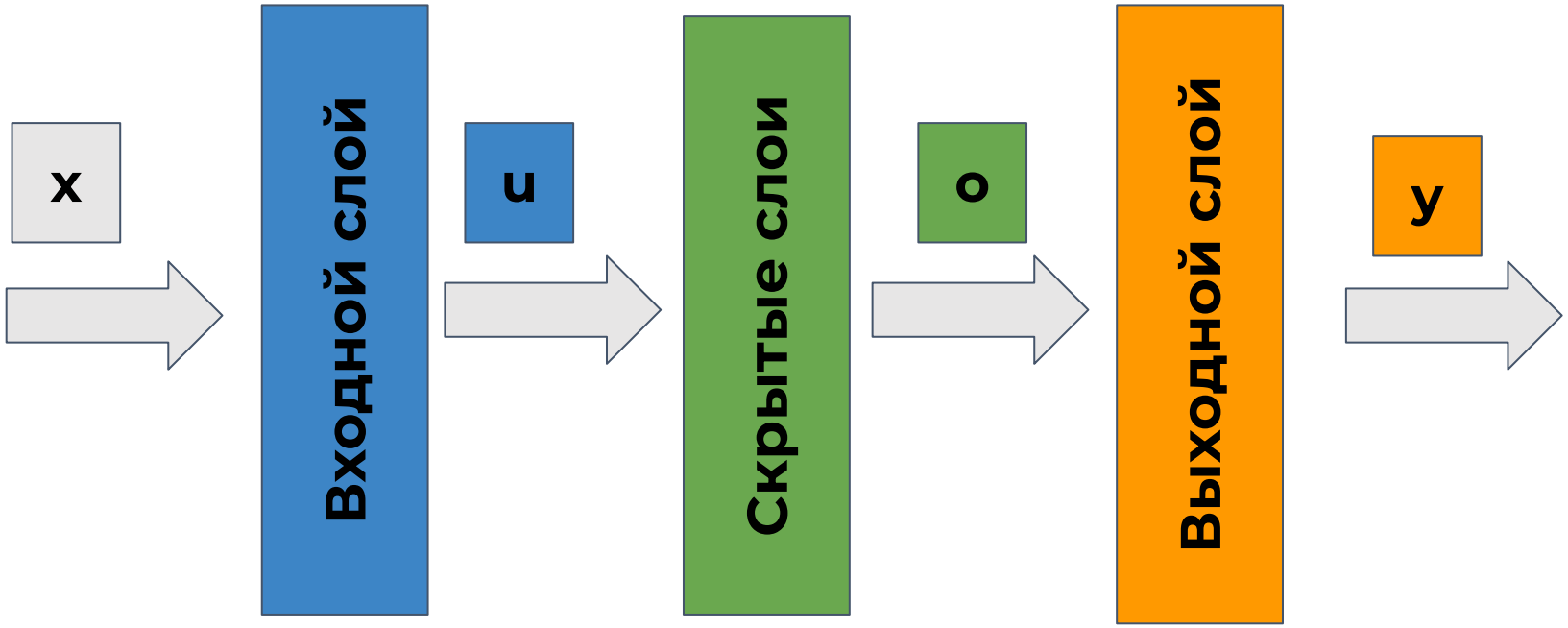
КУРС «НЕЙРОННЫЕ СЕТИ»

Тема 7. RNN, LSTM, GRU

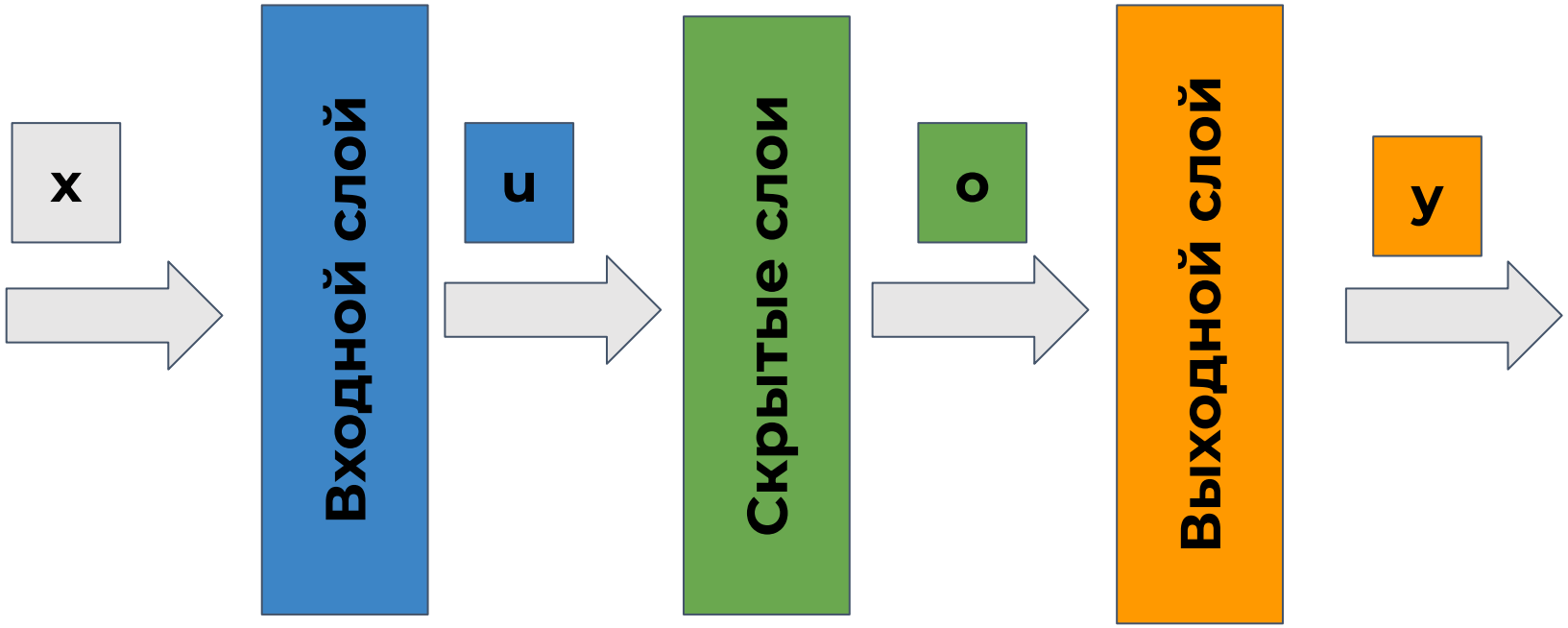


Лекция 7 · Модуль 1

Сети прямого распространения

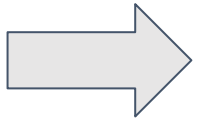


Сети прямого распространения



Сети прямого распространения

x_1 Император
x_2 Петр
x_3 Великий



Модель прямого распространения



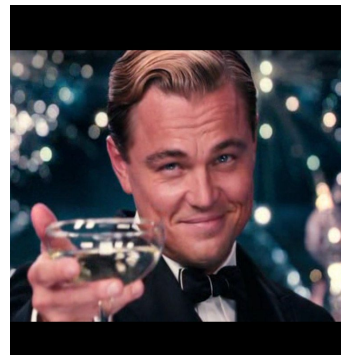
y_1



y_2



y_3



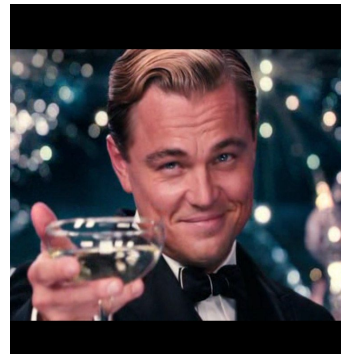
Сети прямого распространения

x_1 Император
x_2 Петр
x_3 Великий

Модель прямого распространения

y_1 y_2 y_3





x_1 Император
x_2 Петр
x_3 Великий

Модель

y_1 $y_2 \mid y_1$ $y_3 \mid y_2, y_1$



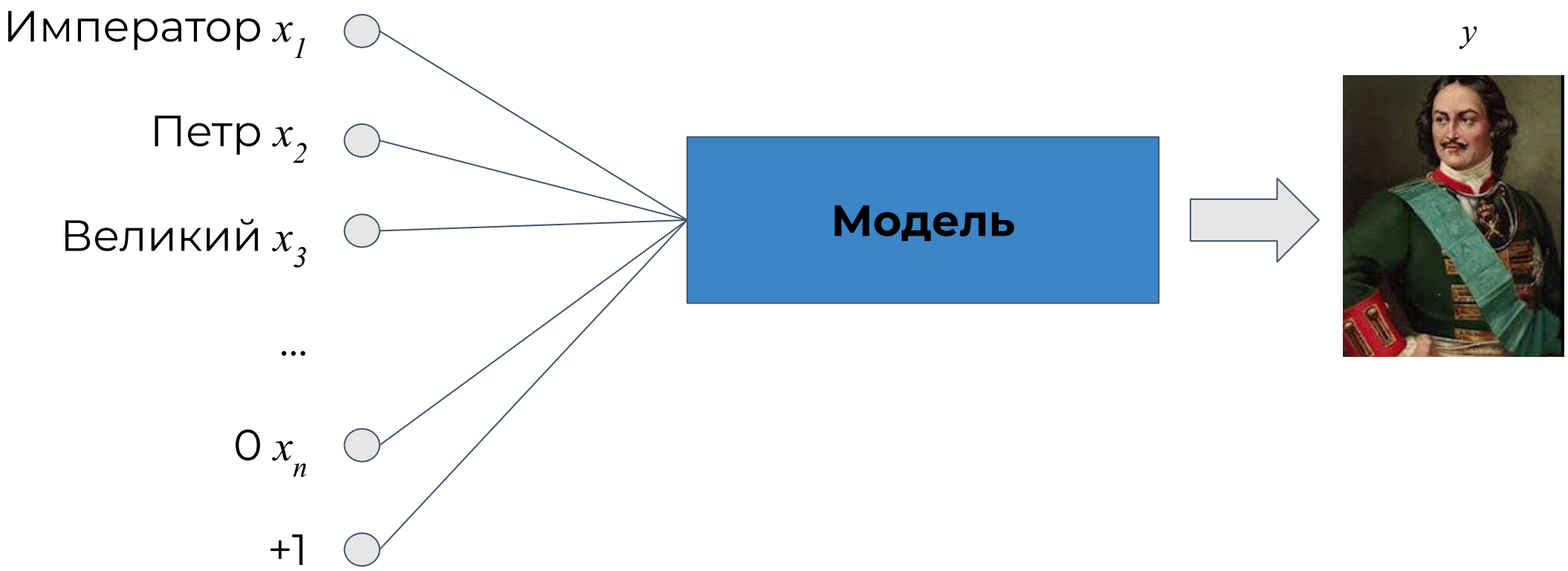


Император

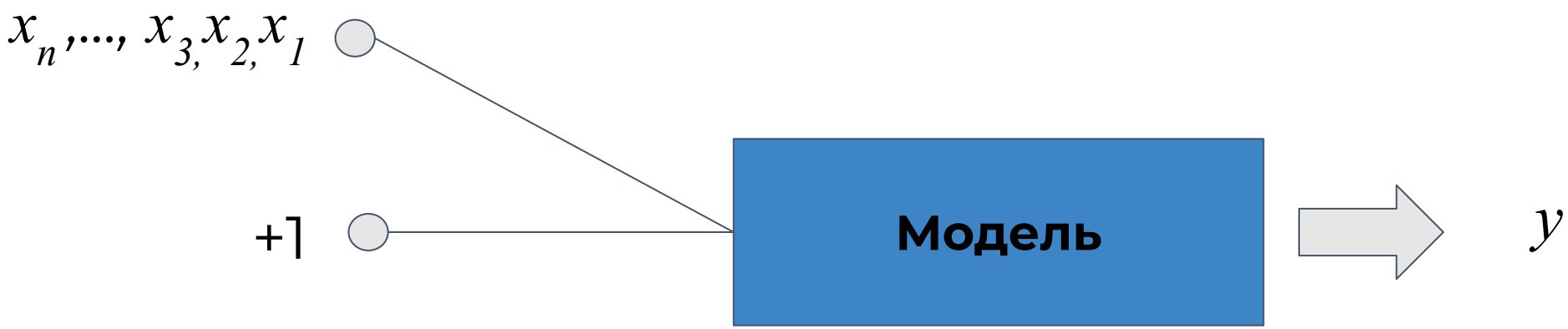
Император
Петр

Император
Петр Великий

Сети прямого распространения

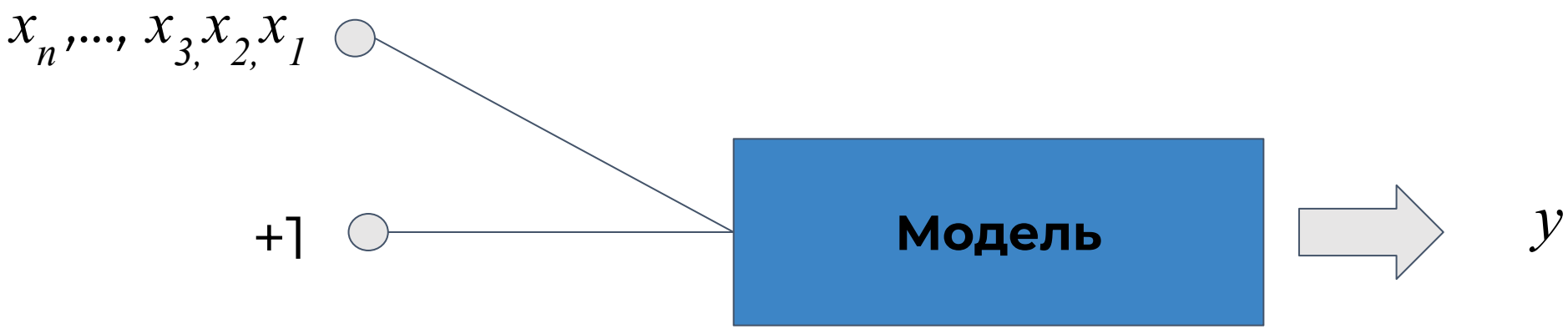


Способ обработки последовательностей



$$h_k = f(x_k, h_{k-1})$$

Способ обработки последовательностей



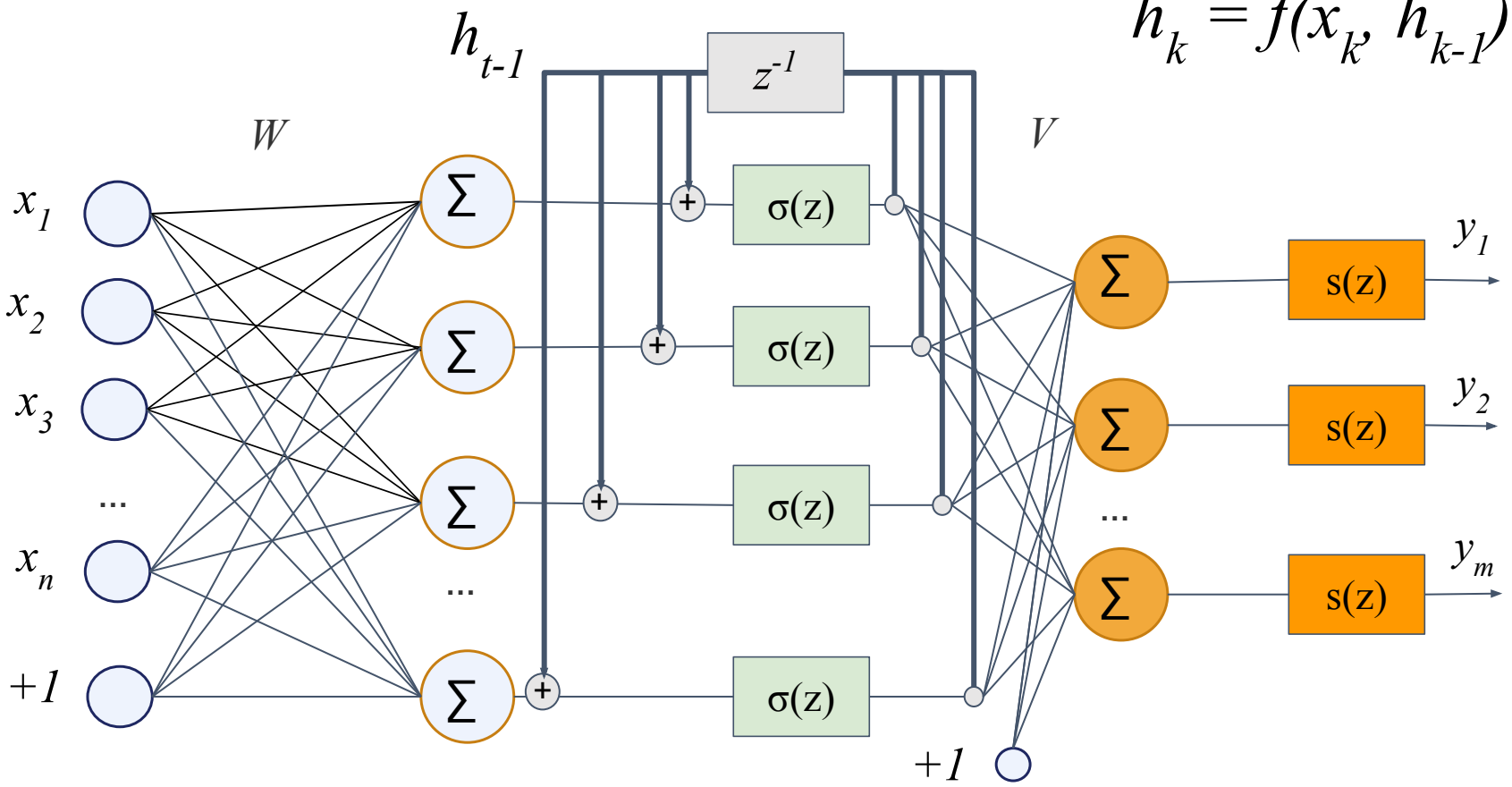
$$h_k = f(x_k, h_{k-1})$$

$$h_k = f(x_k, h_{k-1}) = \alpha(x_k, h_{k-1})$$

$$h_k = (((x_1 + 0)\,\alpha + x_2)\,\alpha + x_3)\alpha = \alpha^3 x_1 + \alpha^2 x_2 + \alpha x_3$$

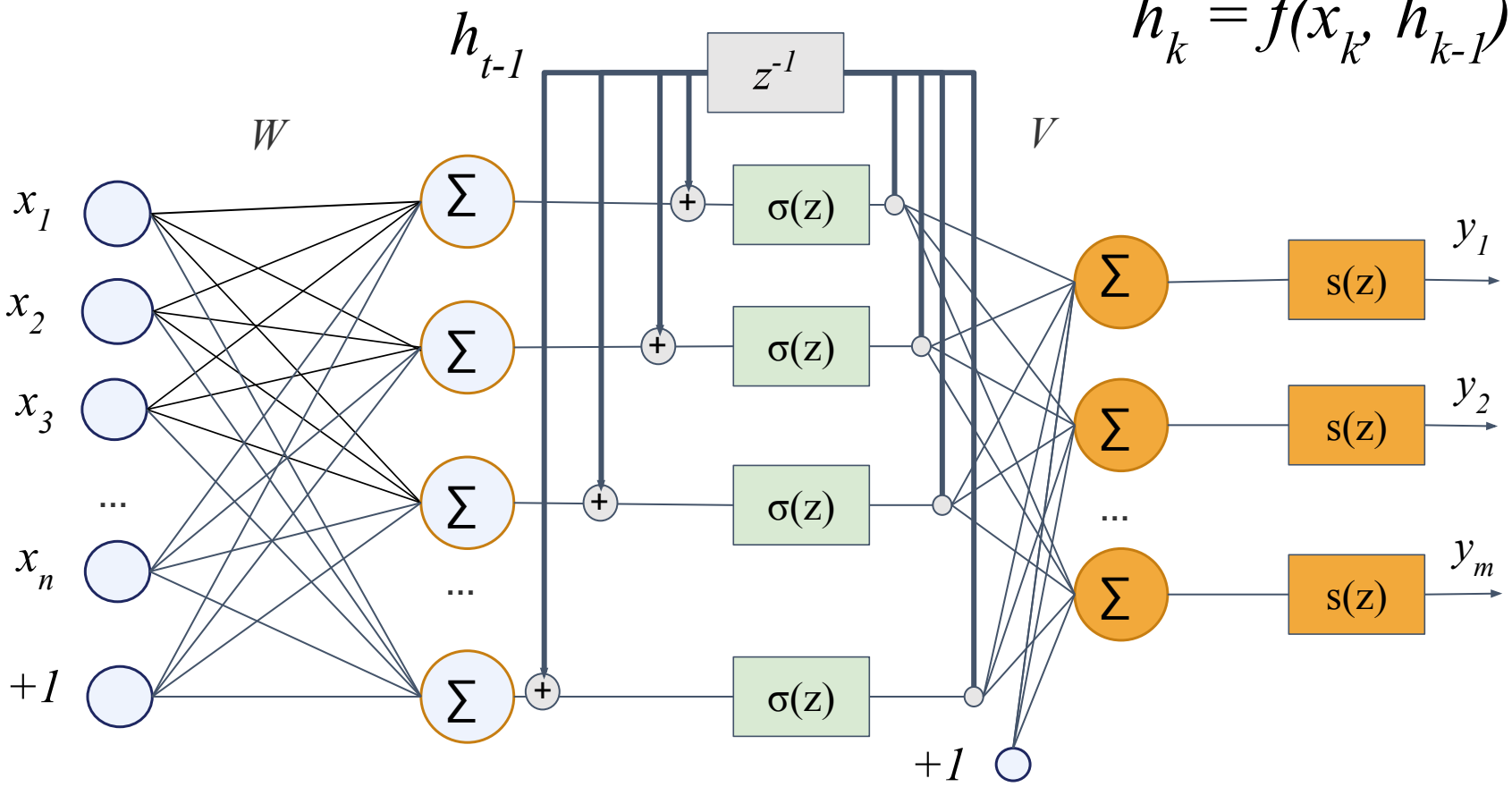
Рекуррентная нейронная сеть

$$h_k = f(x_k, h_{k-1})$$



Рекуррентная нейронная сеть

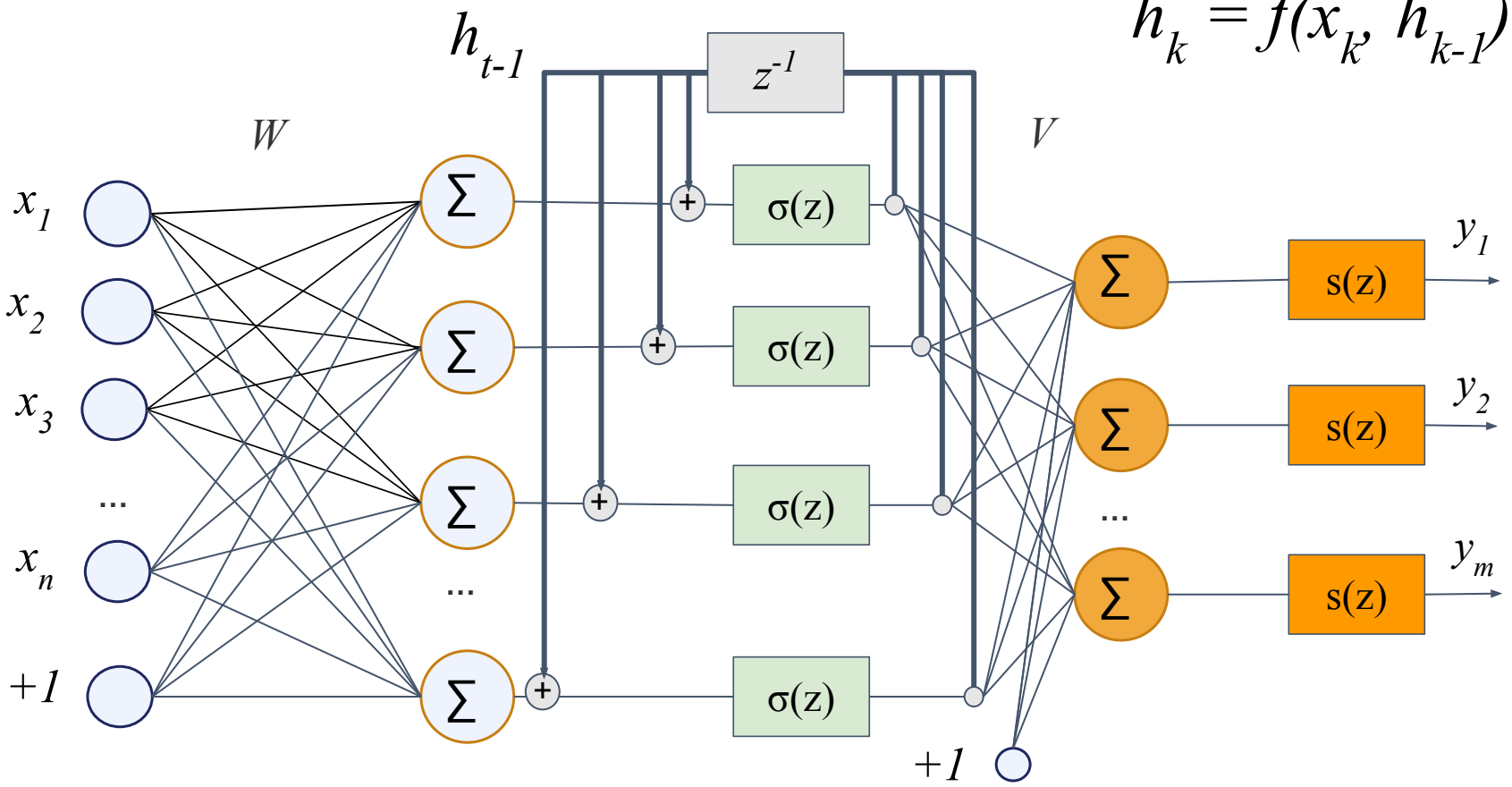
$$h_k = f(x_k, h_{k-1})$$



$$a_t = W x_t + h_{t-1} + b \qquad h_t = \sigma(a_t)$$

Рекуррентная нейронная сеть

$$h_k = f(x_k, h_{k-1})$$

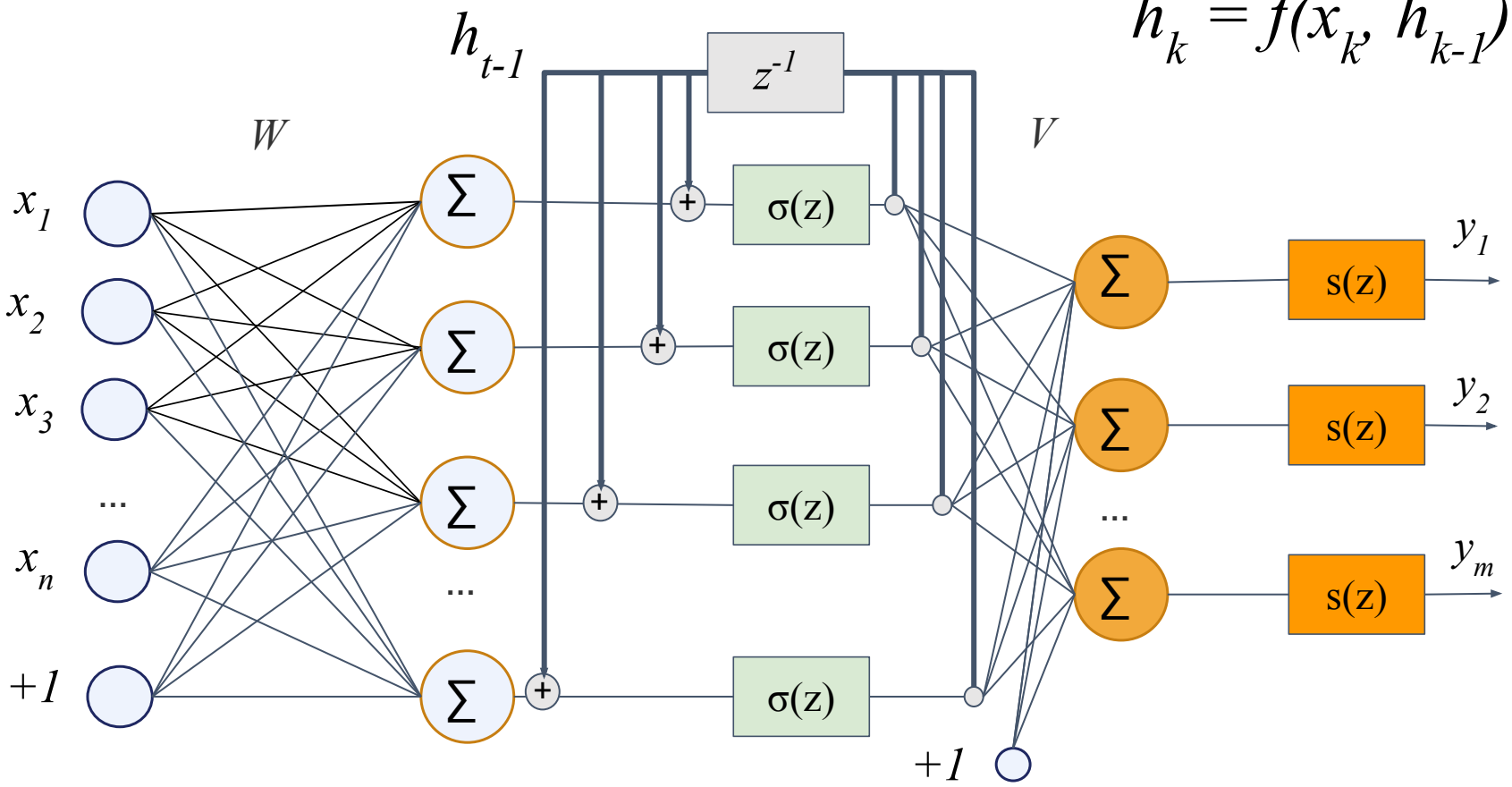


$$a_t = W x_t + h_{t-1} + b \qquad h_t = \sigma(a_t) \qquad o_t = V h_t + c$$

$$y_t = s(o_t)$$

Рекуррентная нейронная сеть

$$h_k = f(x_k, h_{k-1})$$



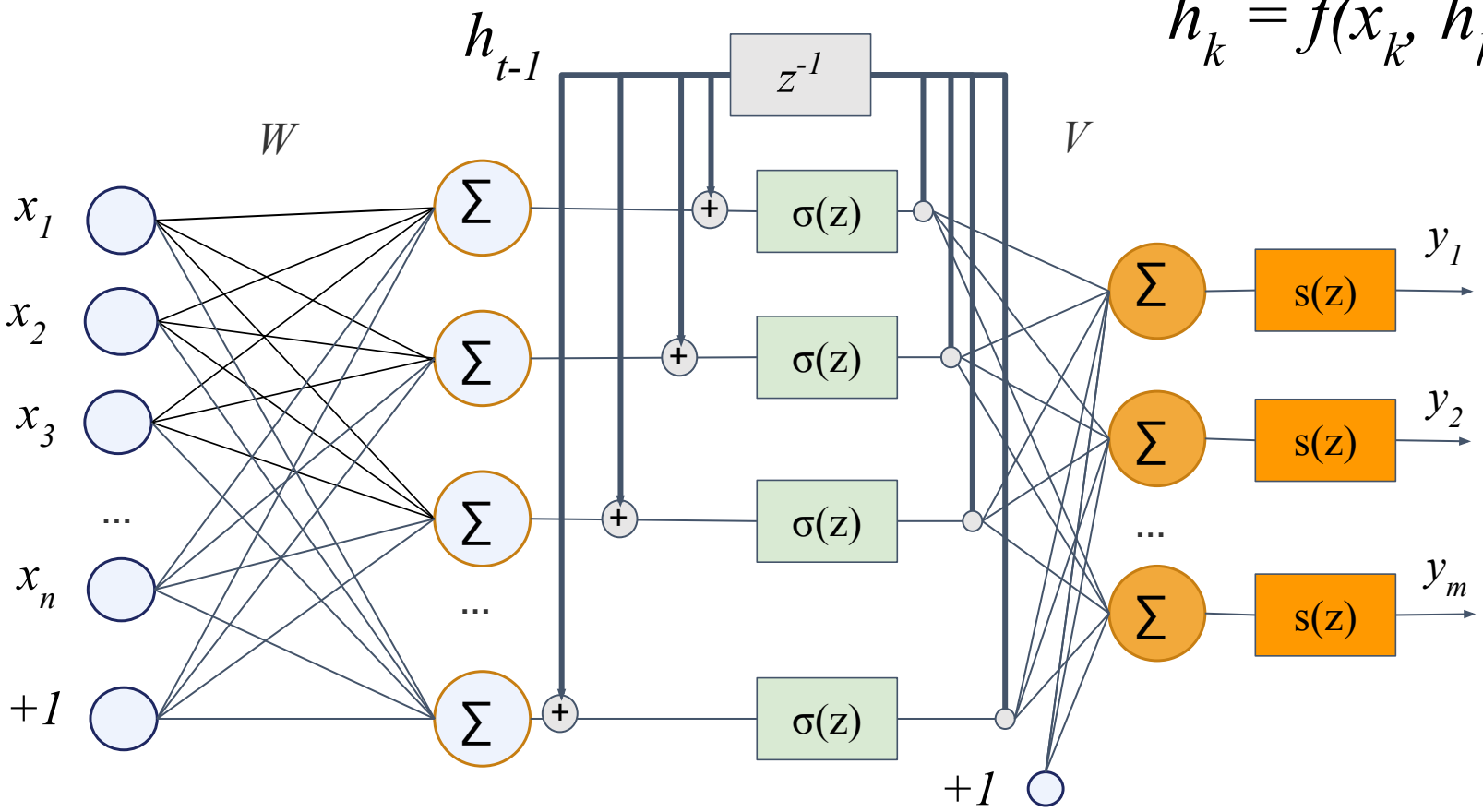
$$a_t = W x_t + h_{t-1} + b \qquad h_t = \sigma(a_t) \qquad o_t = V h_t + c$$

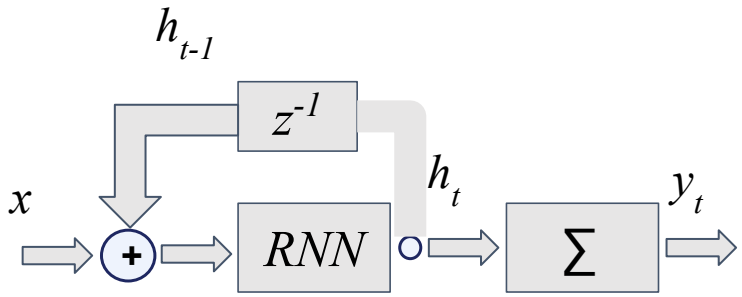
$$y_t = s(o_t)$$

$$h_0 = [0, 0, 0, \ldots, 0_n]$$

Рекуррентная нейронная сеть

$$h_k = f(x_k, h_{k-1})$$



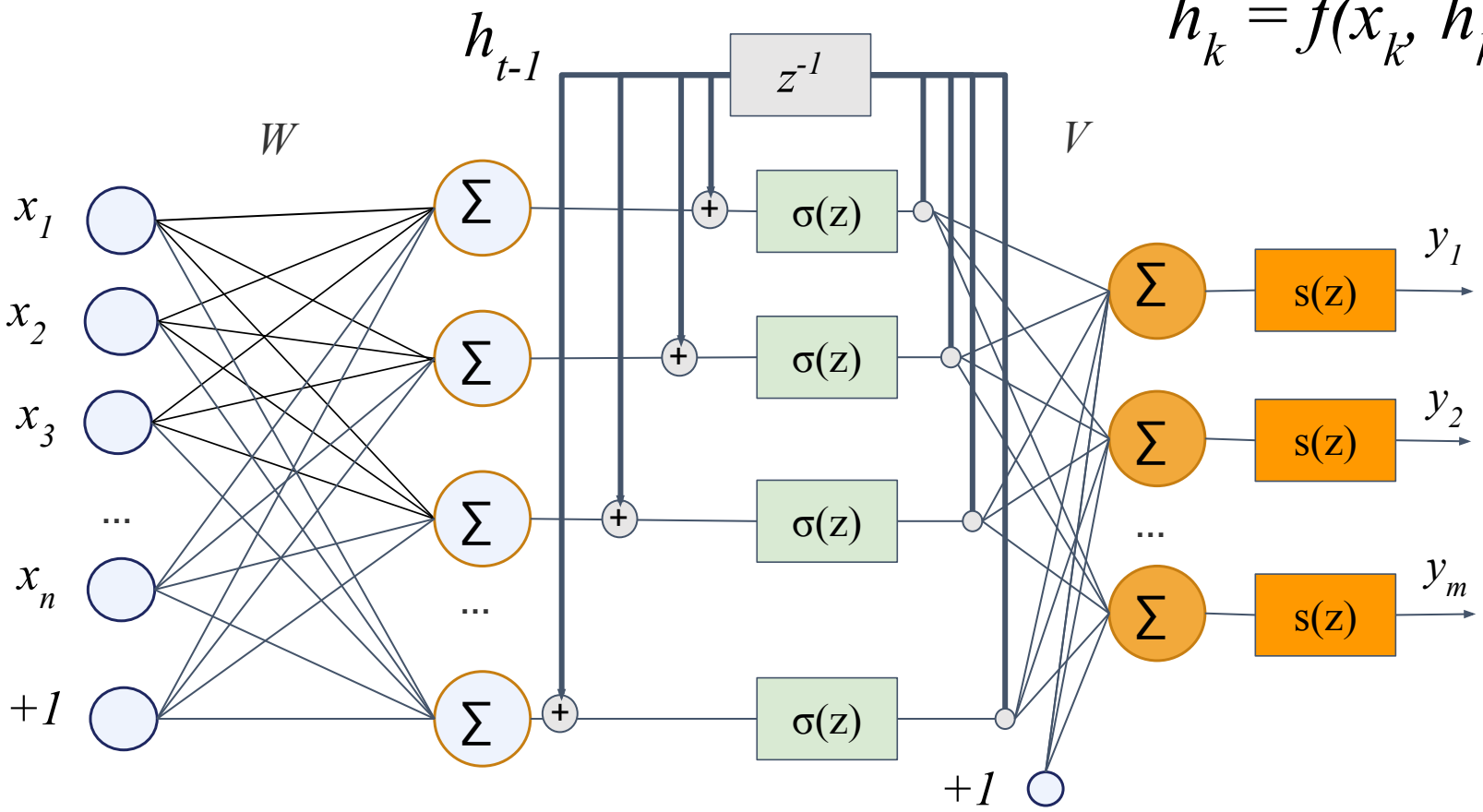


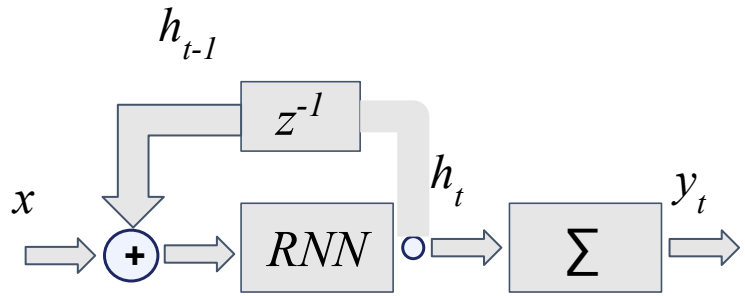
$$a_t = W x_t + h_{t-1} + b \qquad h_t = \sigma(a_t) \qquad o_t = V h_t + c$$

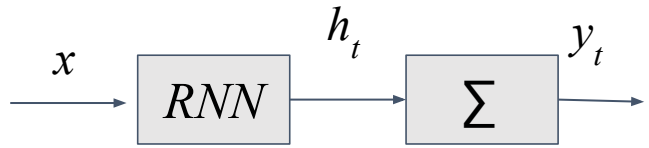
$$h_0 = [0, 0, 0, \ldots, 0_n] \qquad y_t = s(o_t)$$

Рекуррентная нейронная сеть

$$h_k = f(x_k, h_{k-1})$$





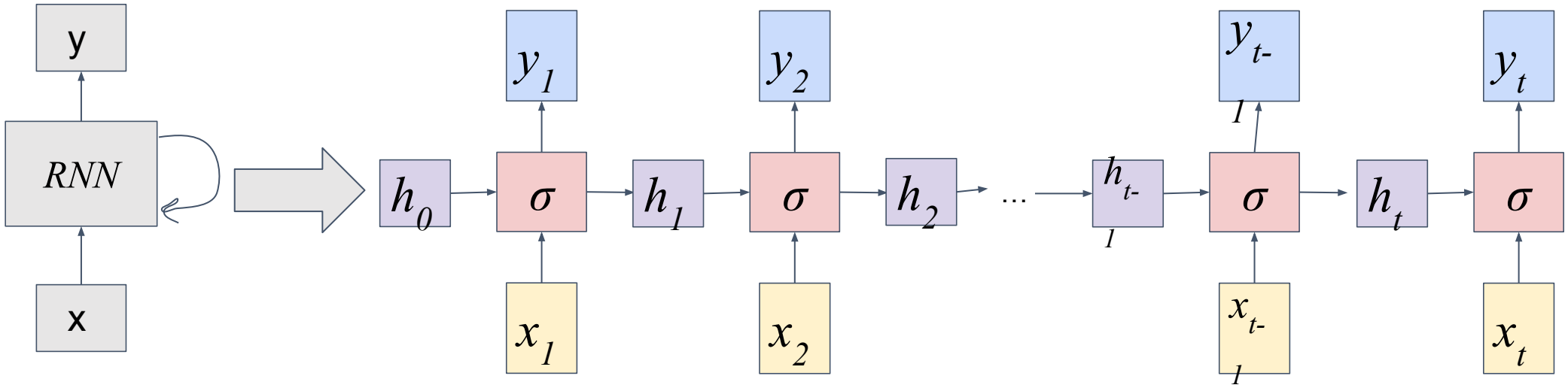


$$a_t = W x_t + h_{t-1} + b \qquad h_t = \sigma(a_t) \qquad o_t = V h_t + c$$

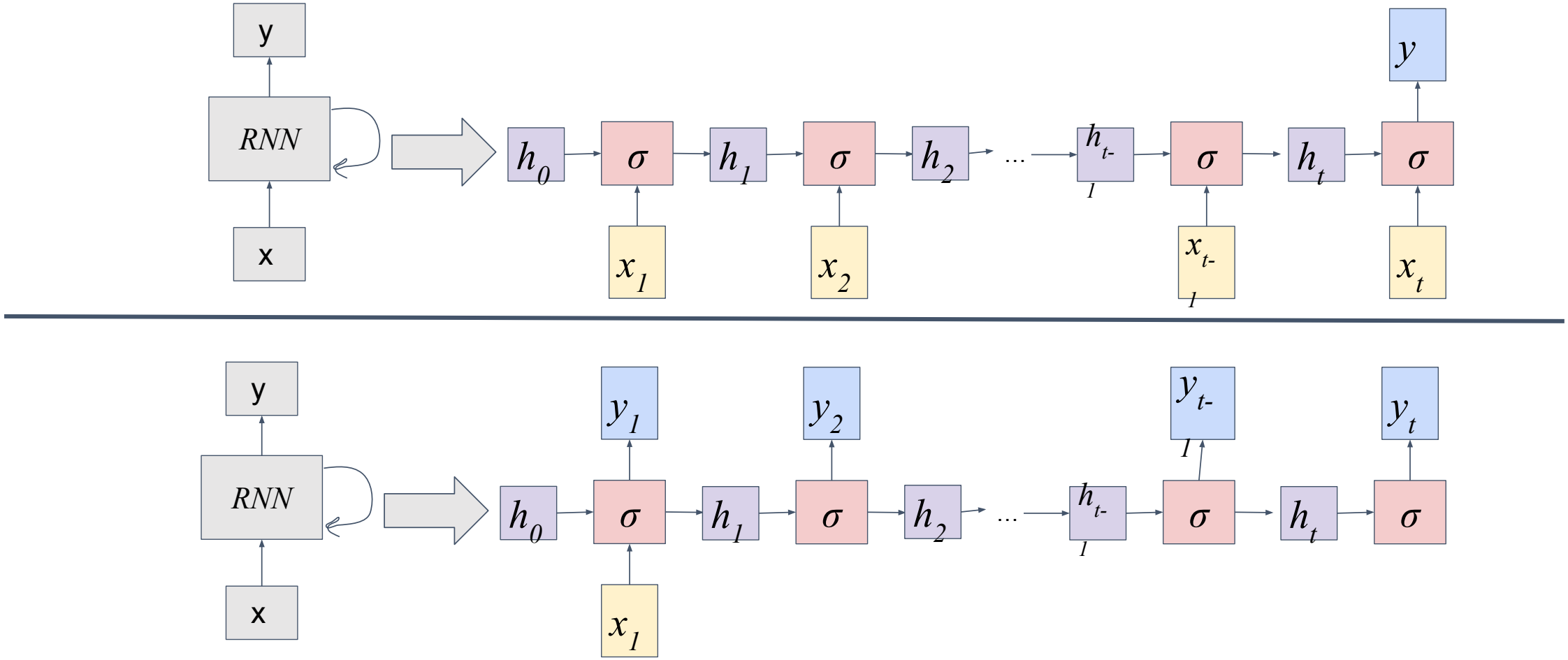
$$y_t = s(o_t)$$

$$h_0 = [0, 0, 0, \ldots, 0_n]$$

Развертка рекуррентной сети во времени



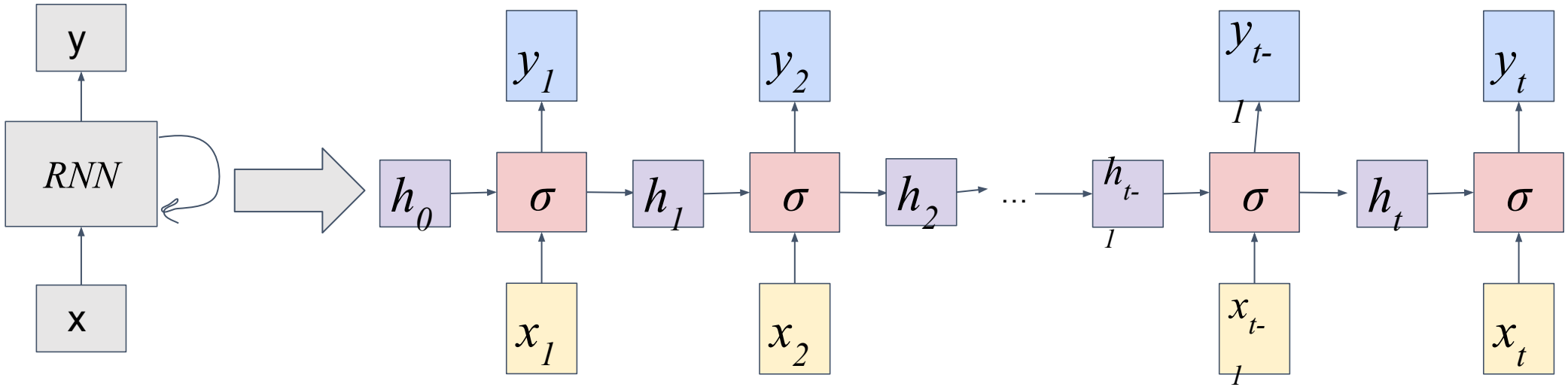
Архитектуры Many-to-One и One-to-Many



Архитектуры рекуррентных сетей

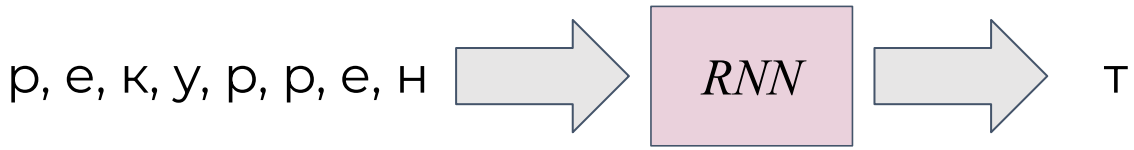
- Many-to-Many - перевод текста
- Many-to-One - анализ эмоциональной окраски текста (на входе текст, на выходе категория - положительный, нейтральный, отрицательный)
- One-to-Many - генерация изображений - на вход карты признаков изображения - на выходе описание (текст)
- One-to-One - Сеть принимает на вход один образец и выдаёт одну метку класса (например, определение спама в электронном письме: «это спам» или «это не спам»)

Развертка рекуррентной сети во времени

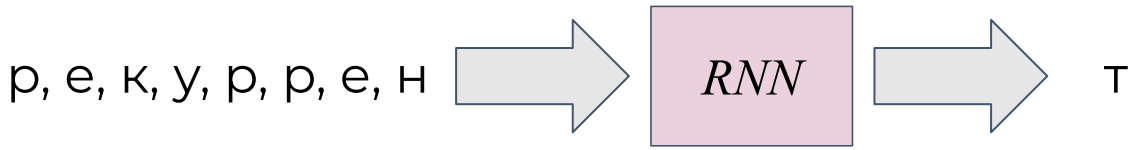


д

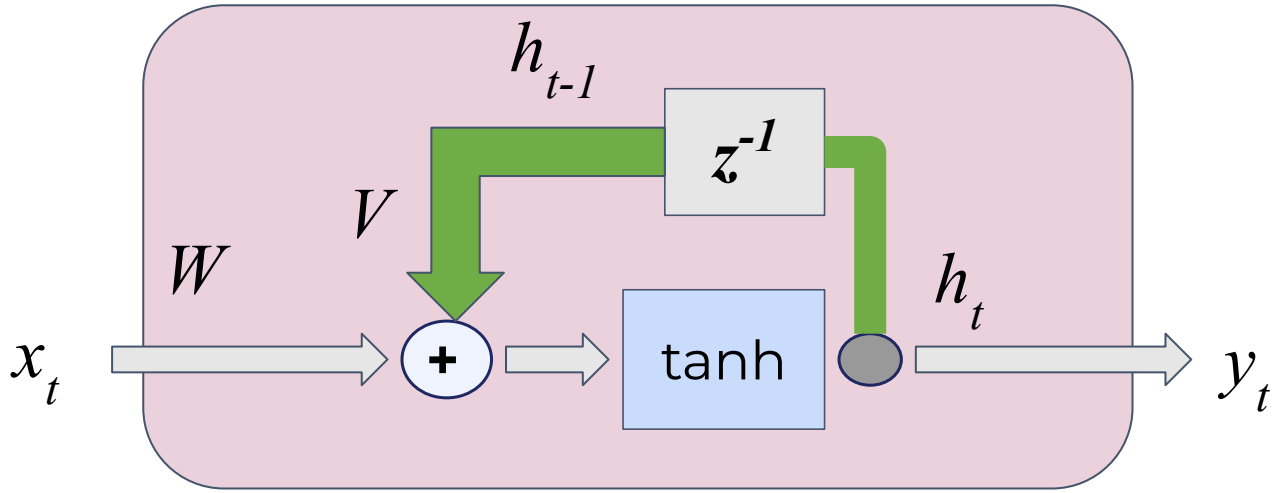
Реализация рекуррентного слоя в PyTorch



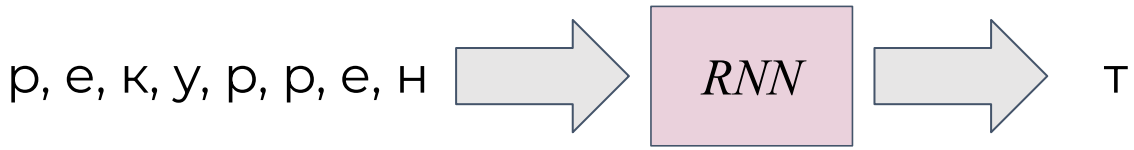
Реализация рекуррентного слоя в PyTorch



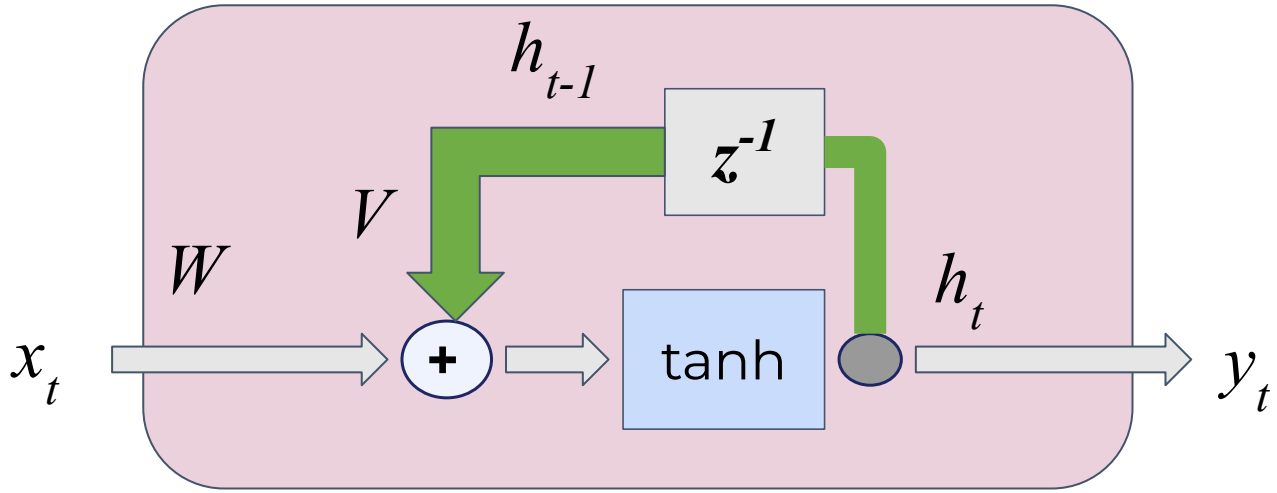
torch.nn.**RNN**



Реализация рекуррентного слоя в PyTorch



torch.nn.**RNN**



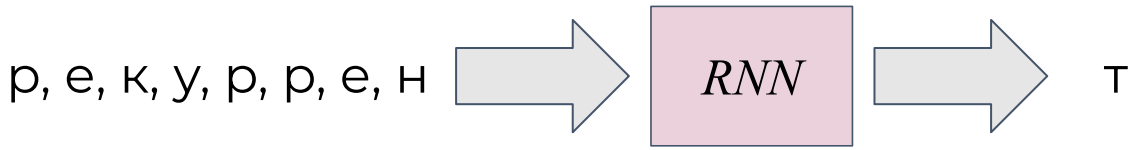
$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

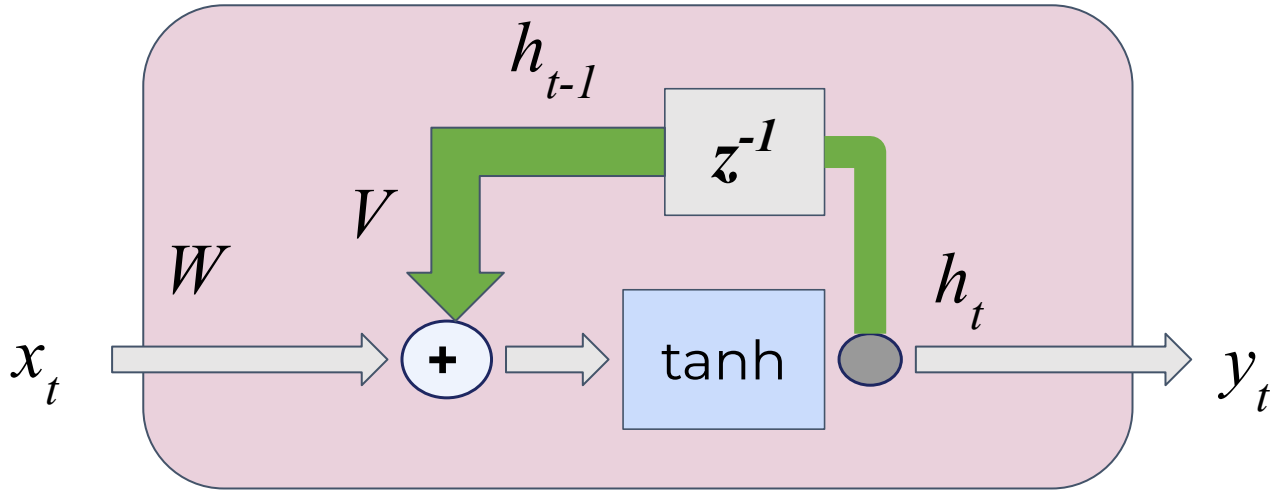
$$y = h_t$$

Реализация рекуррентного слоя в PyTorch

torch.nn.RNN(input_size, hidden_size, num_layers=1,
nonlinearity='tanh', bias=True,
batch_first=False)



torch.nn.**RNN**



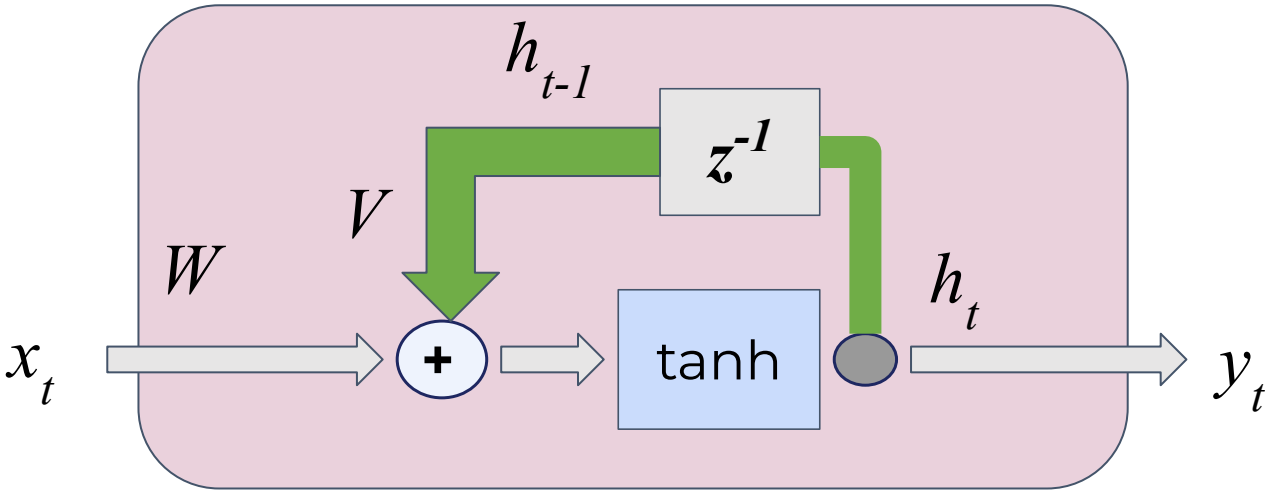
$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

$$y = h_t$$

Реализация рекуррентного слоя в PyTorch

torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)



$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

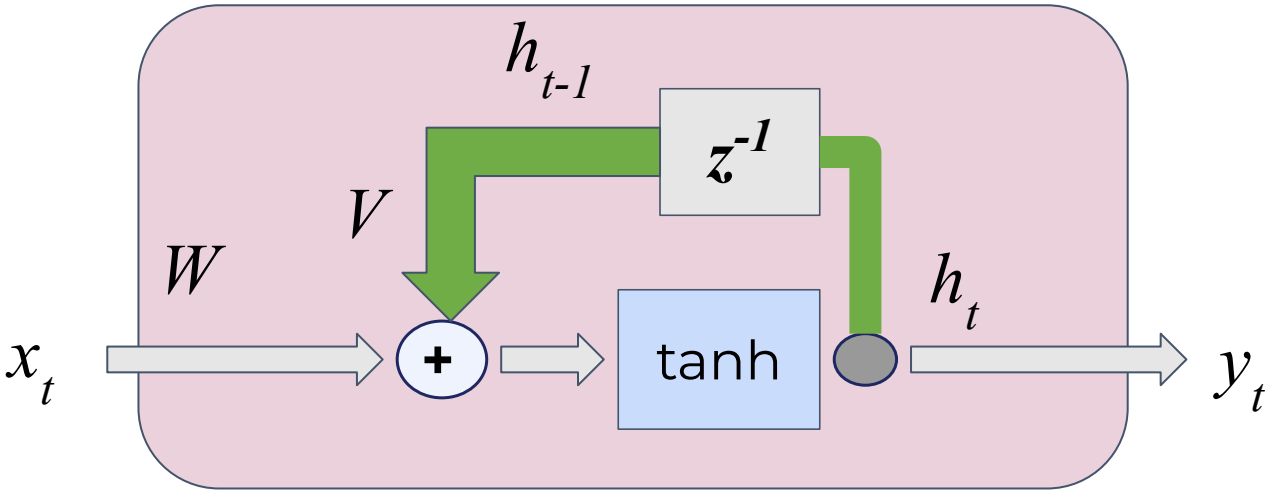
$$y = h_t$$

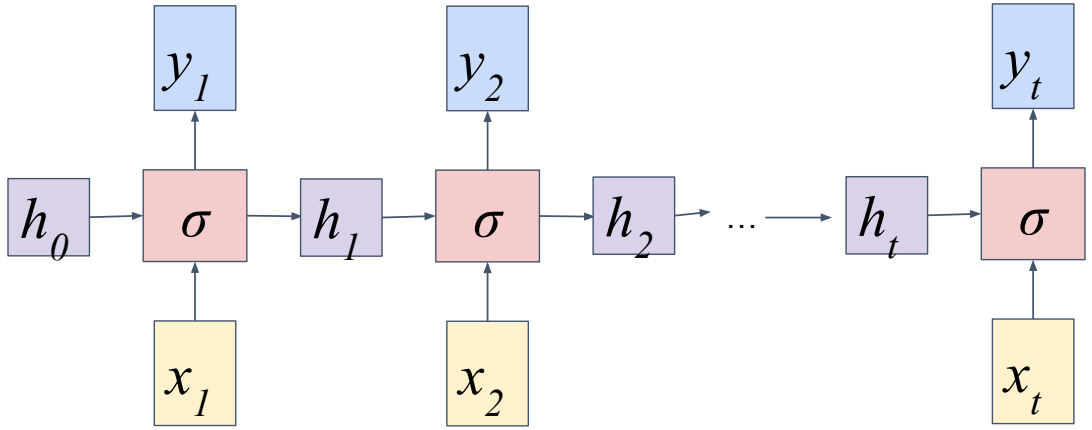
x_t (seq_length, batch_size, d_sieze)

- **seq_length** - длина подаваемой последовательности (число элементов)
- **batch_size** - размер батча
- **d_size** - размер каждого подаваемого элемента последовательности

Исчезающие и взрывающиеся градиенты

torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)





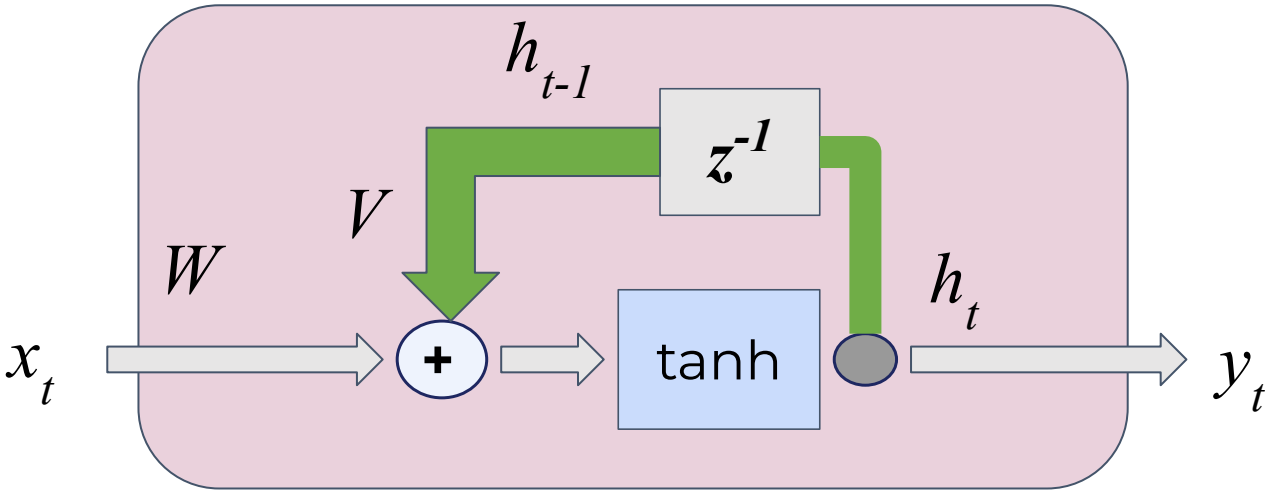
$$a = W x_t + V h_{t-1} + b_x + b_n$$

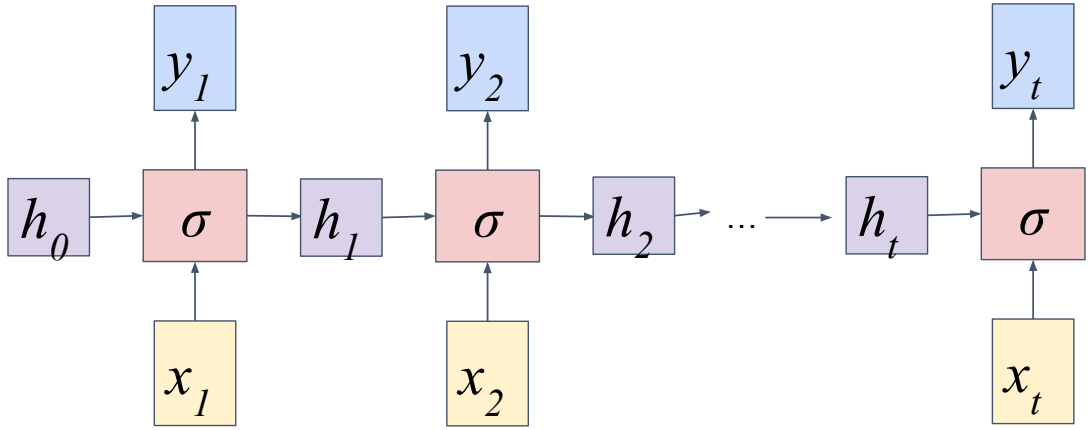
$$h_t = tanh(a_t)$$

$$y = h_t$$

Исчезающие и взрывающиеся градиенты

torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)





$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

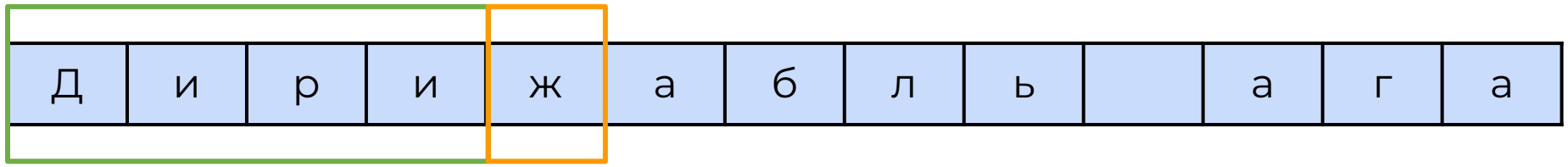
$$y = h_t$$

LSTM, GRU

$$\delta_j = \begin{cases} \delta_j, & |\delta_j| < \lambda \\ \lambda, & |\delta_j| \geq \lambda \end{cases}$$

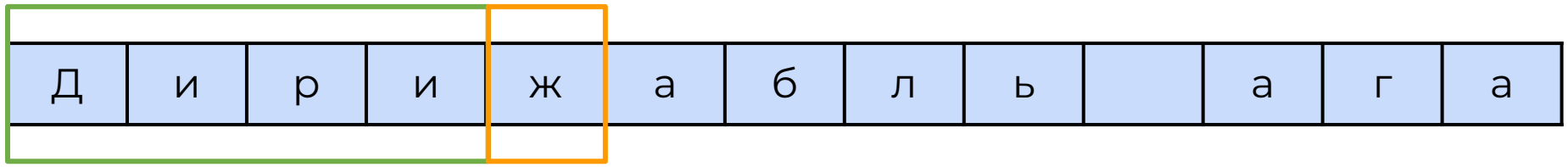
Формирование обучающей выборки

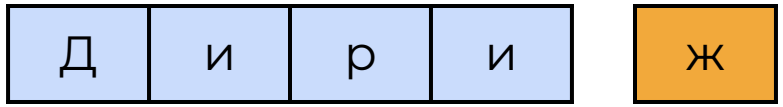
Предыдущие символы



Формирование обучающей выборки

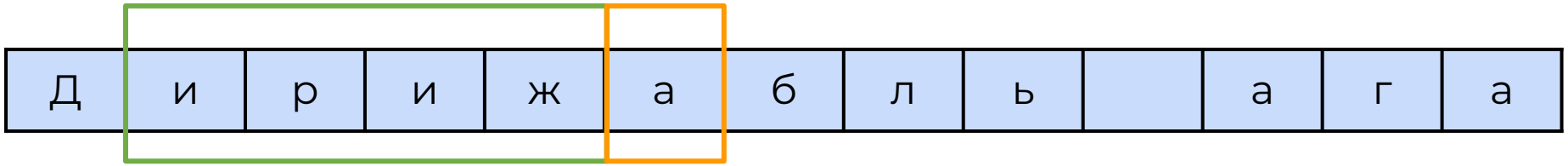
Предыдущие символы





Формирование обучающей выборки

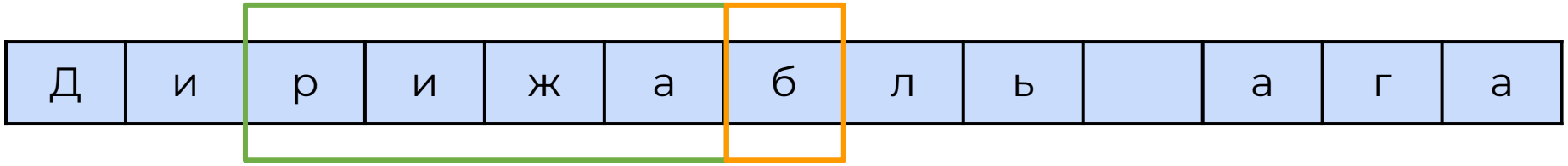
Предыдущие символы



Д	и	р	и	ж
и	р	и	ж	а

Формирование обучающей выборки

Предыдущие символы

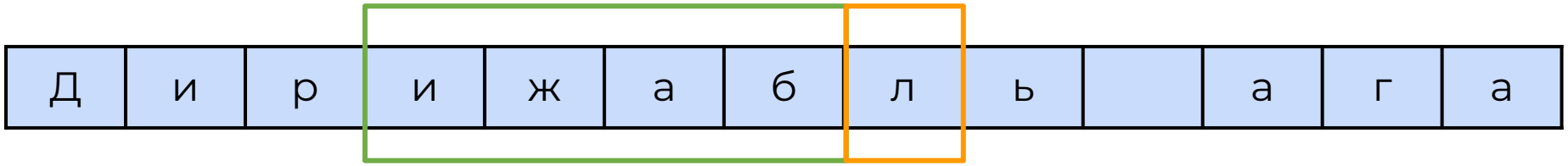


Д	и	р	и
и	р	и	ж
р	и	ж	а

ж
а
б

Формирование обучающей выборки

Предыдущие символы

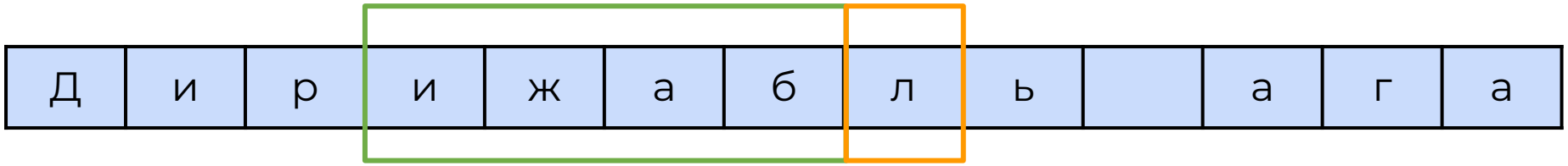


Д	и	р	и	ж
и	р	и	ж	а
р	и	ж	а	б
и	ж	а	б	л

Many-to-One

Формирование обучающей выборки

Предыдущие символы

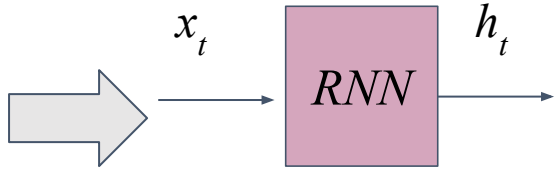


Д	и	р	и
и	р	и	ж
р	и	ж	а
и	ж	а	б

ж
а
б
л

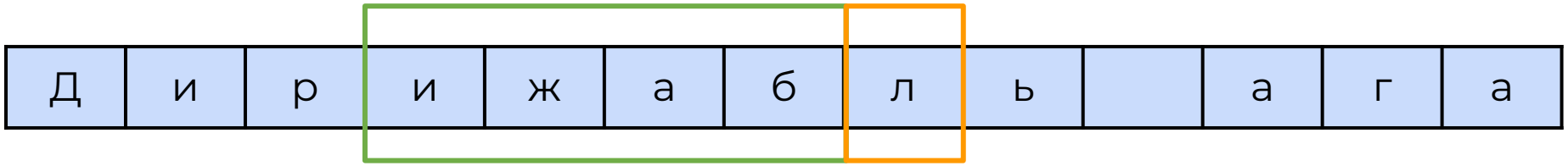
Many-to-One

а: 0
б: 1
в: 2
…
я: 32
“: 33



Формирование обучающей выборки

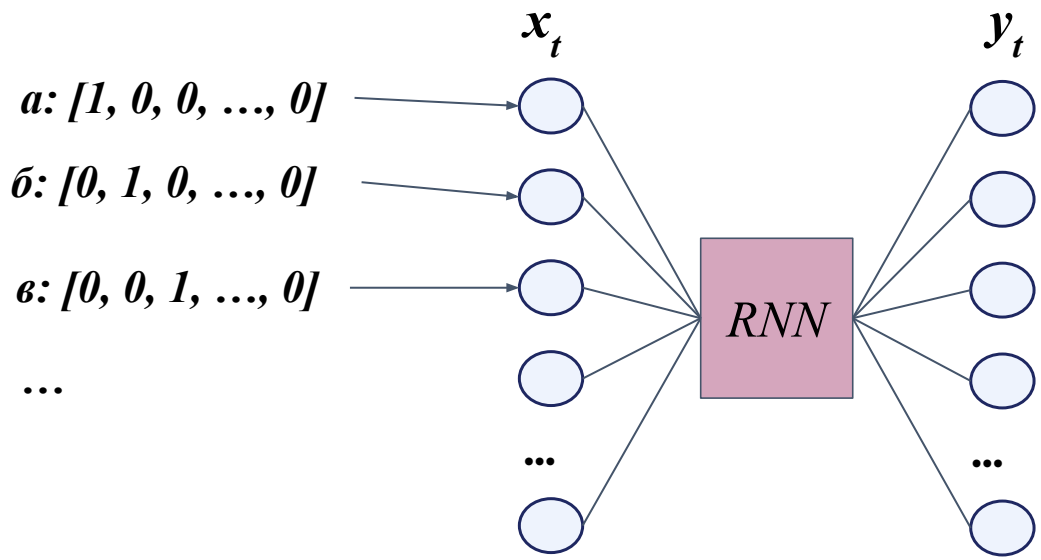
Предыдущие символы



Д	и	р	и
и	р	и	ж
р	и	ж	а
и	ж	а	б

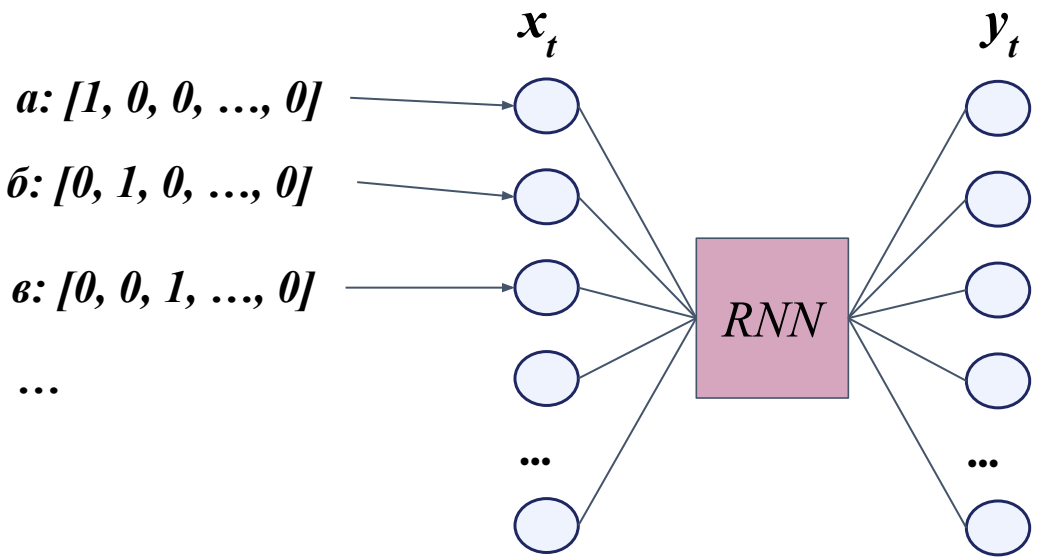
ж
а
б
л

Many-to-One



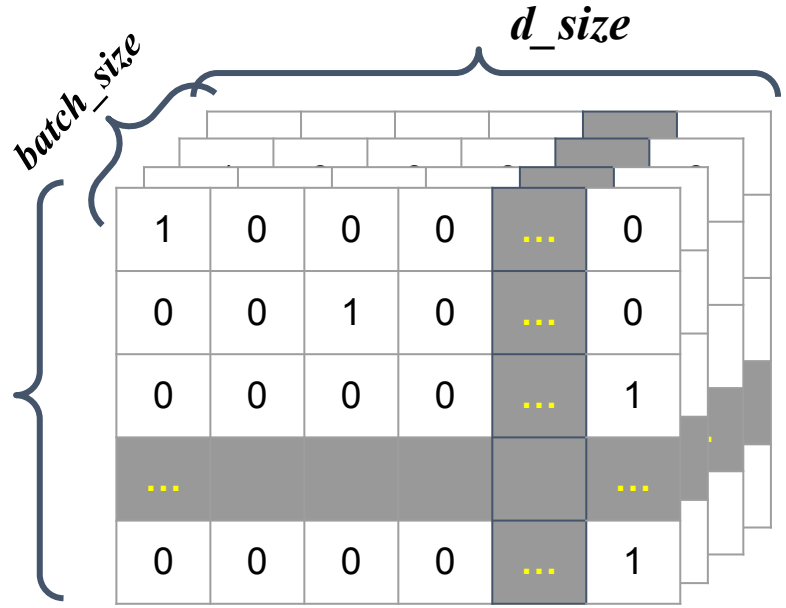
One Hot Encoding

Формирование обучающей выборки



Д	и	р	и
и	р	и	ж
р	и	ж	а
и	ж	а	б

ж
а
б
л

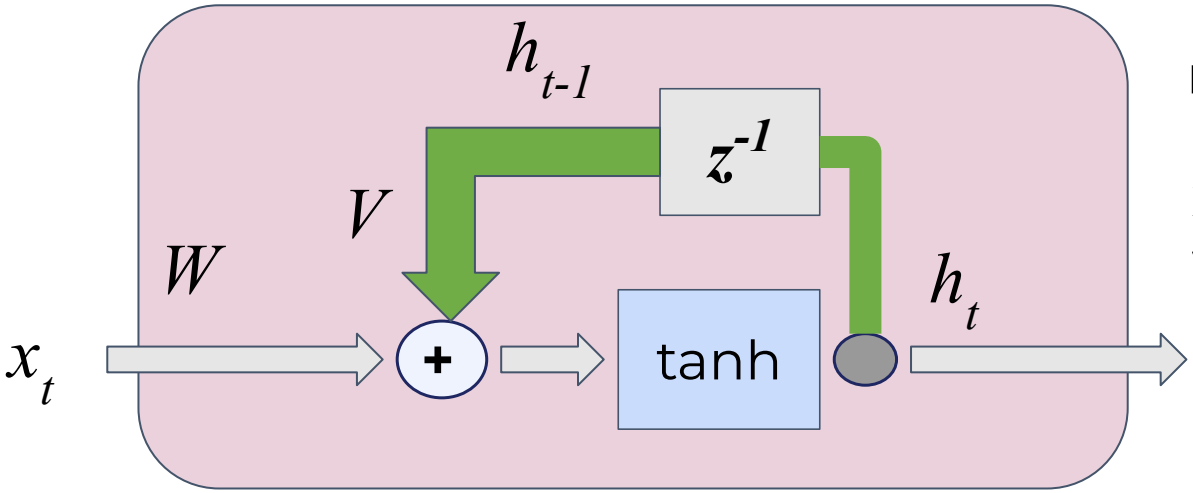


x_i *(batch_size, seq_length, d_size)*

seq_length

Класс torch.nn.RNN

torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)



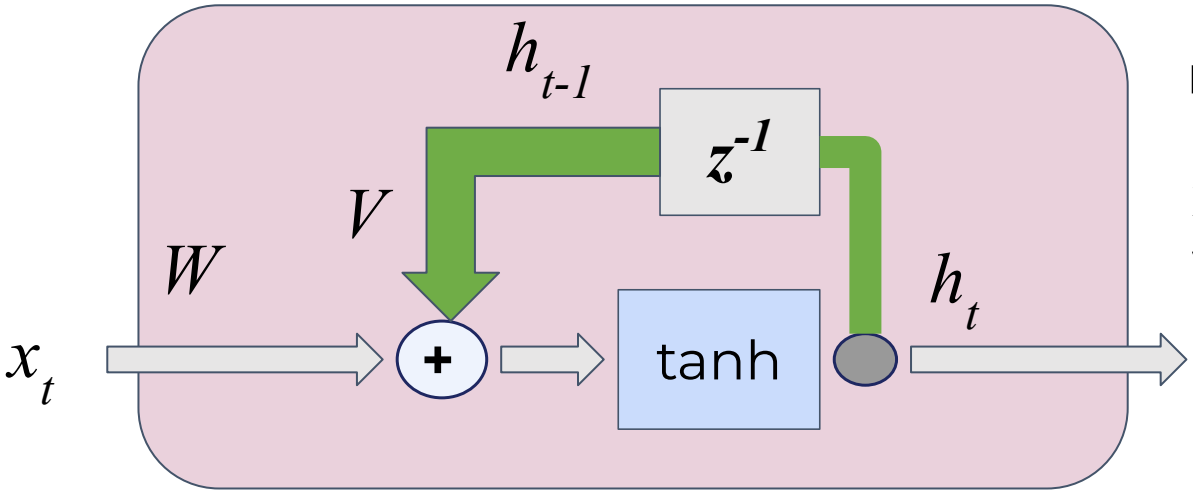

```
rnn = nn.RNN(d_size, hidden_size, batch_first=True)

x = torch.randn(8, 3, 33)
y, h = rnn(x)
```

x_i *(batch_size, seq_length, d_size)*

Класс torch.nn.RNN

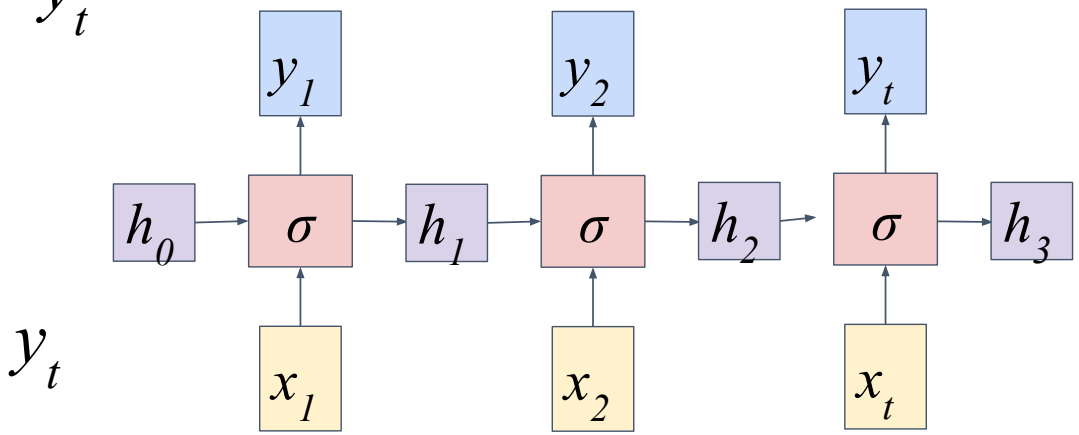
torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)



x_i *(batch_size, seq_length, d_size)*

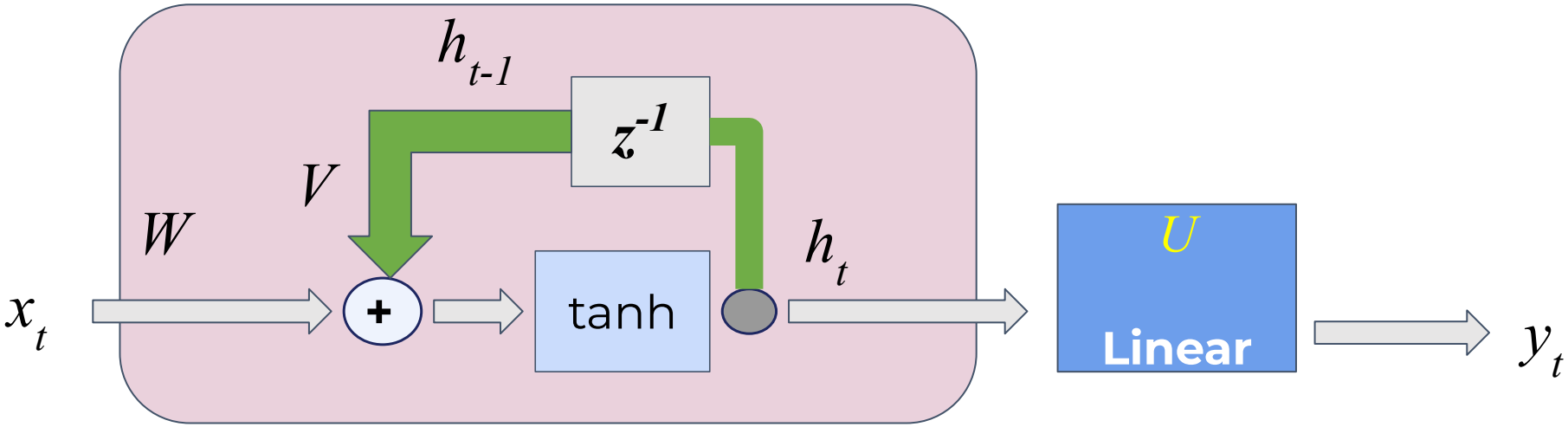
```
rnn = nn.RNN(d_size, hidden_size, batch_first=True)

x = torch.randn(8, 3, 33)
y, h = rnn(x)
```



Модель рекурентной нейронной сети

torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)

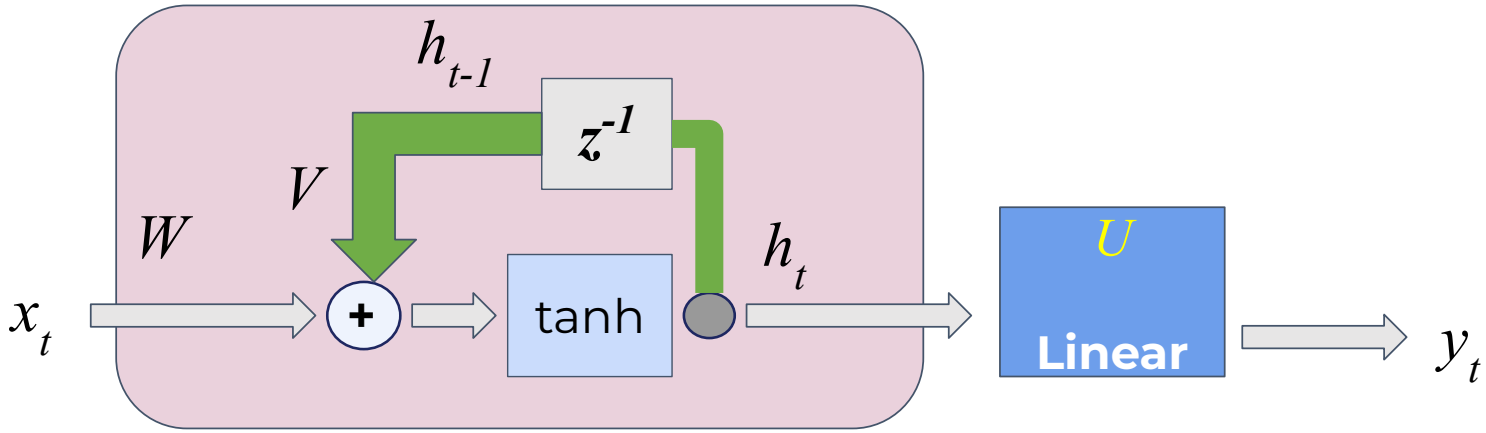


$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

$$y = U h_t + b_y$$

Модель рекурентной нейронной сети

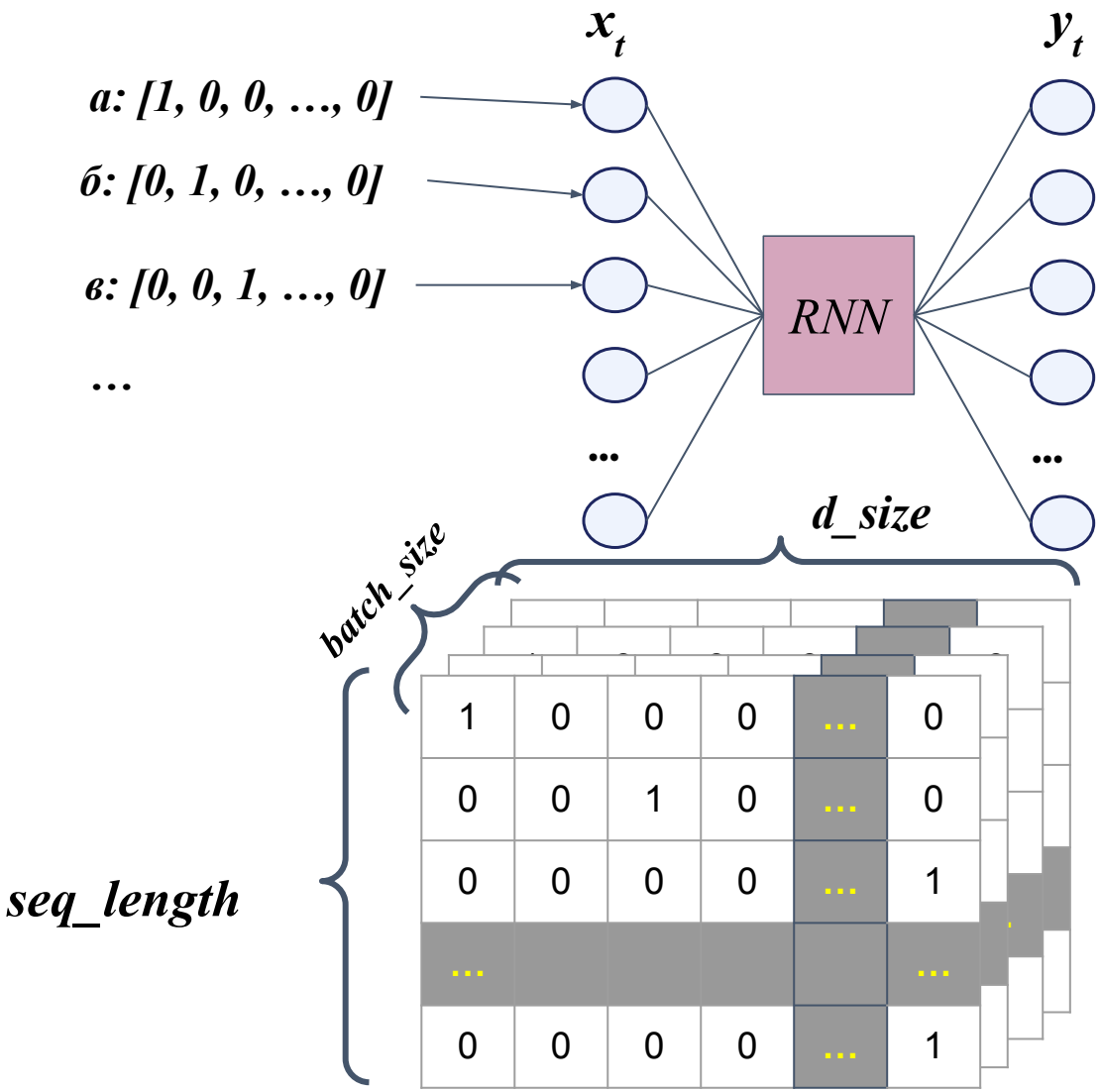


$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

$$y = U h_t + b_y$$

Формирование обучающей выборки



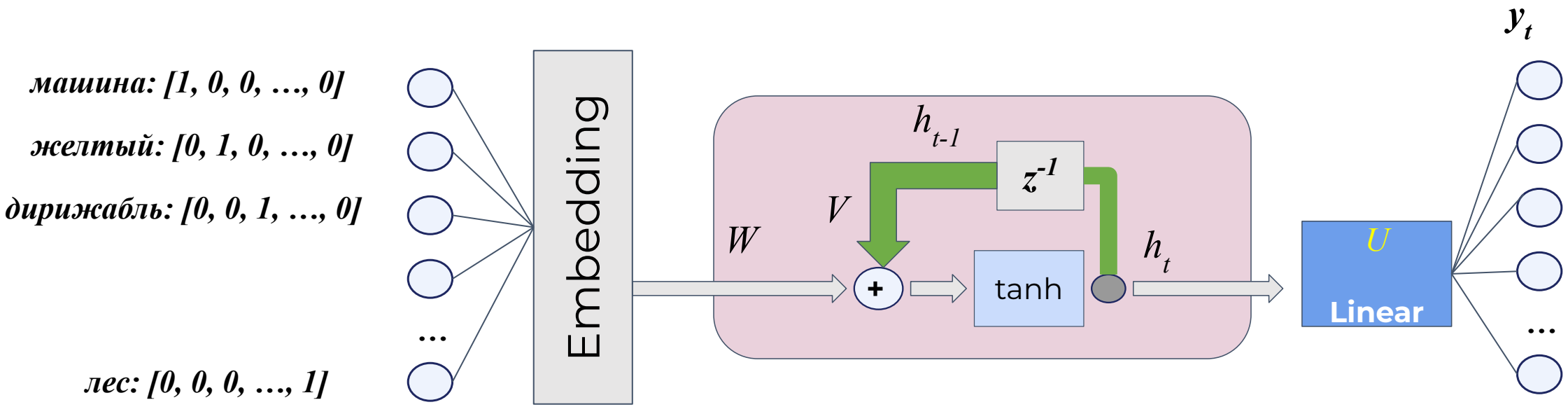

```
class CharsDataset(data.Dataset):
    def __init__(self, prev_chars=3):
        self prev_chars = prev_chars

    with open(path, 'r', encoding='utf-8') as f:
        self.text = f.read()
        self.text = self.text.replace('\ufeff', )
        self.text = re.sub(r'[^A-яA-z0-9.,?;: ]', '', self.text)

        self.text = self.text.lower()
        self.alphabet = set(self.text)
        self.int_to_alpha = dict(enumerate(sorted(self.alphabet)))
        self.num_characters = len(self.alphabet)
        self.onehots = torch.eye(self.num_characters)

    def __getitem__(self, item):
        _data = torch.vstack([self.onehots[self.alpha_to_int[self.text[x]]]
                              for x in range(item, item+self.prev_chars)])
        ch = self.text[item+self.prev_chars]
        t = self.alpha_to_int[ch]
        return _data, t
```

Модель для прогноза слов



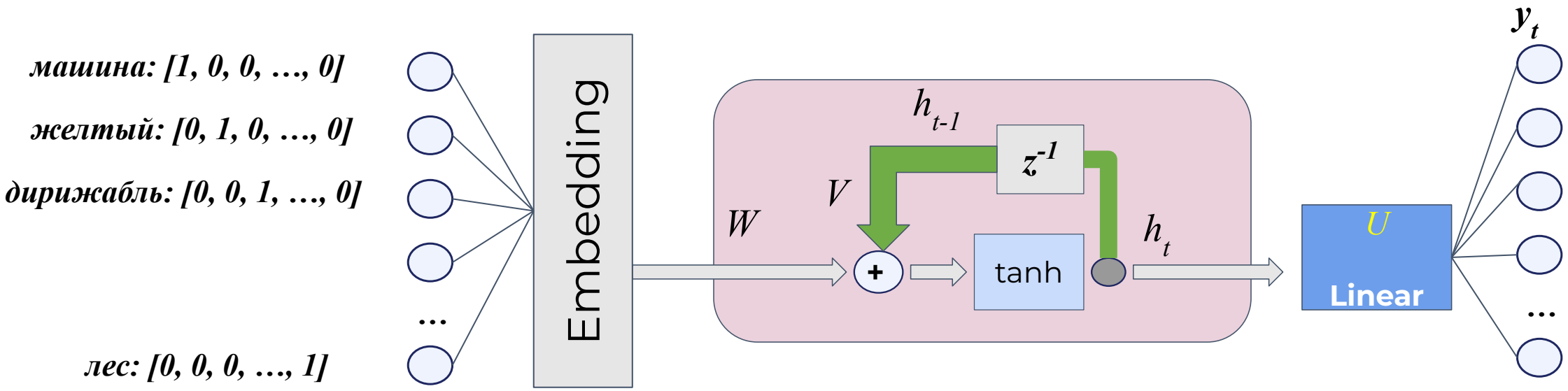
$$e_t = Emb * x_t$$

$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

$$y = U h_t + b_y$$

Модель для прогноза слов



torch.nn.Embedding (num_embeddings, embedding_dim, ...)

- num_embedding - размер словаря
- embedding_dim - размер вектора эмбедингов

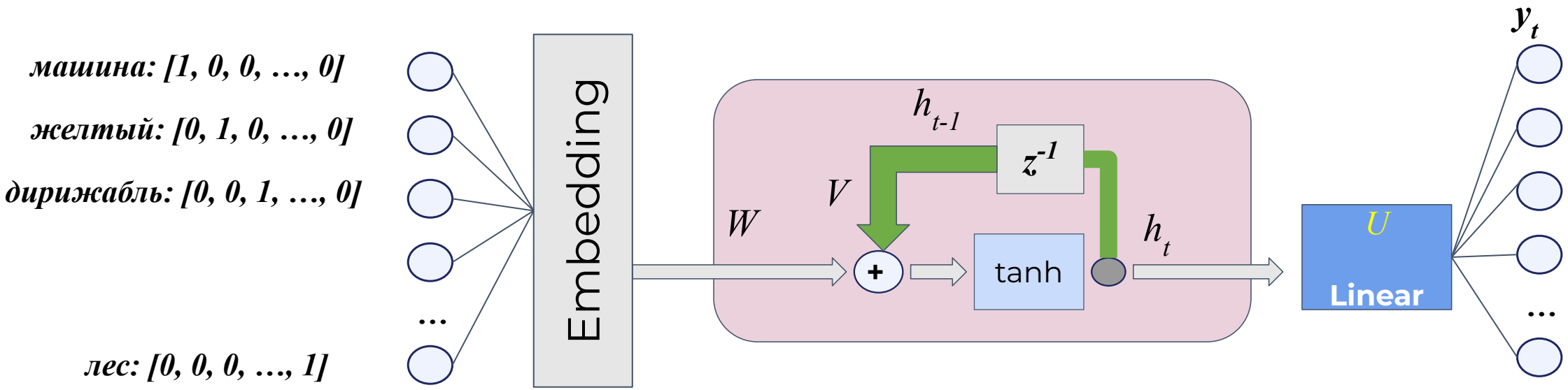
$$e_t = Emb * x_t$$

$$a = W x_t + V h_{t-1} + b_x + b_n$$

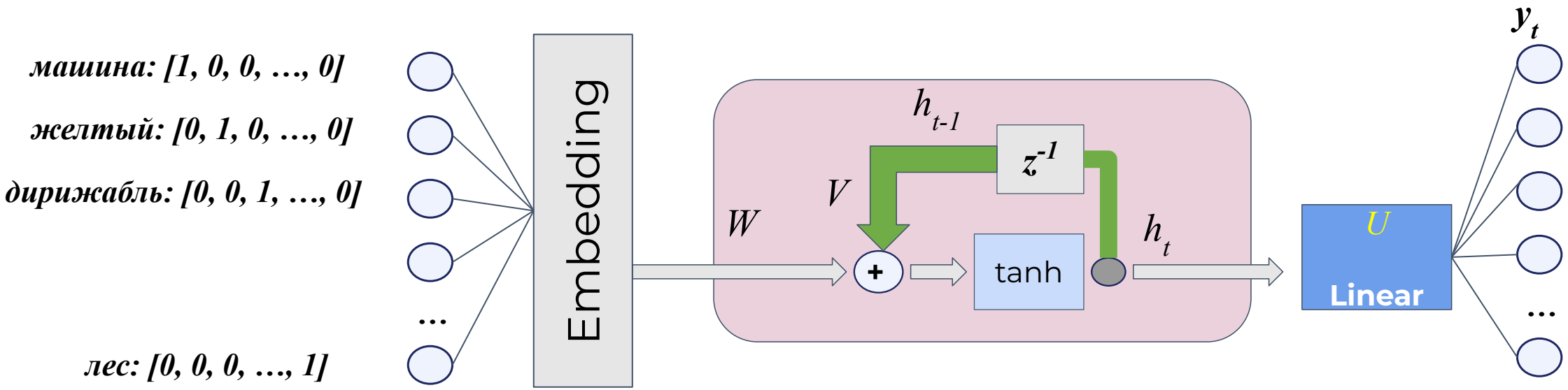
$$h_t = tanh(a_t)$$

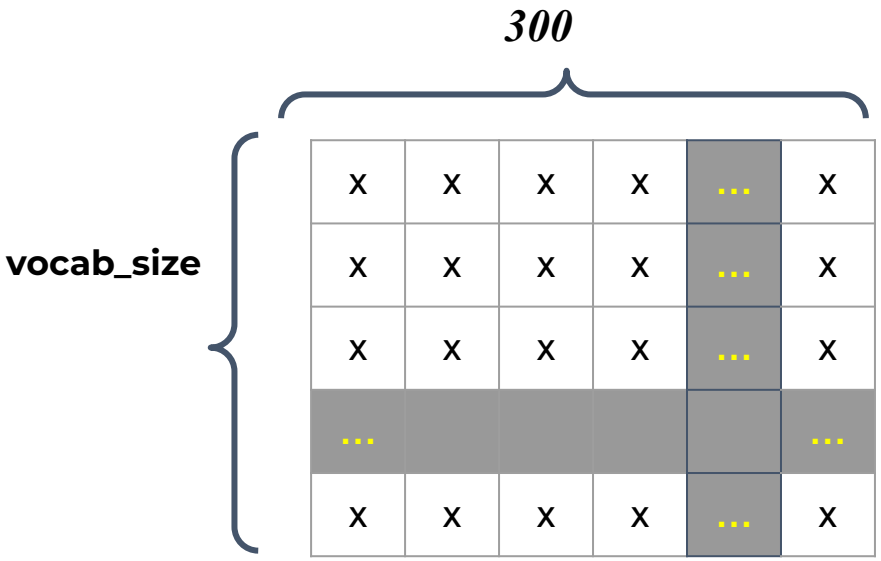
$$y = U h_t + b_y$$

Модель для прогноза слов

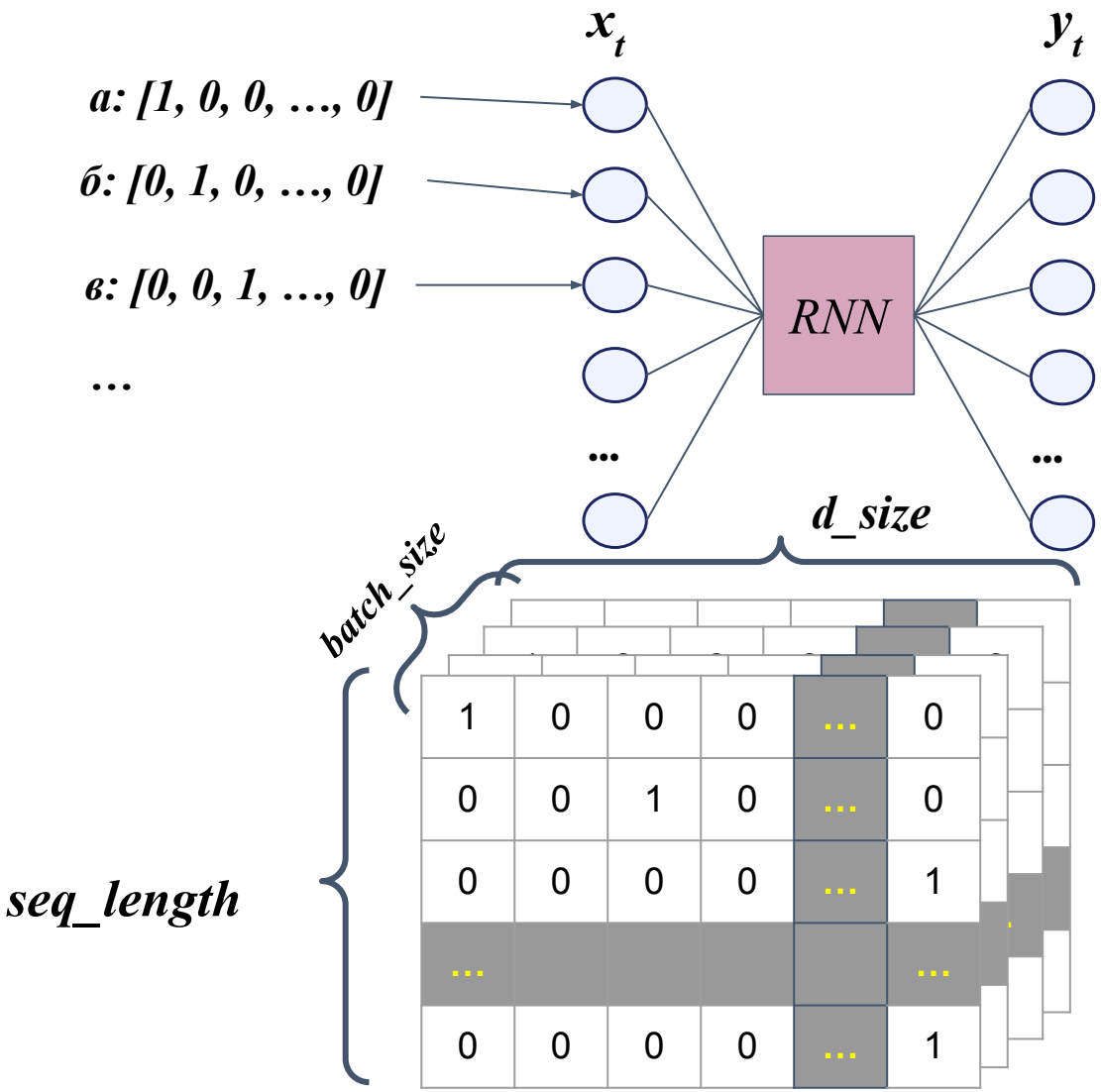


Модель для прогноза слов





Формирование обучающей выборки



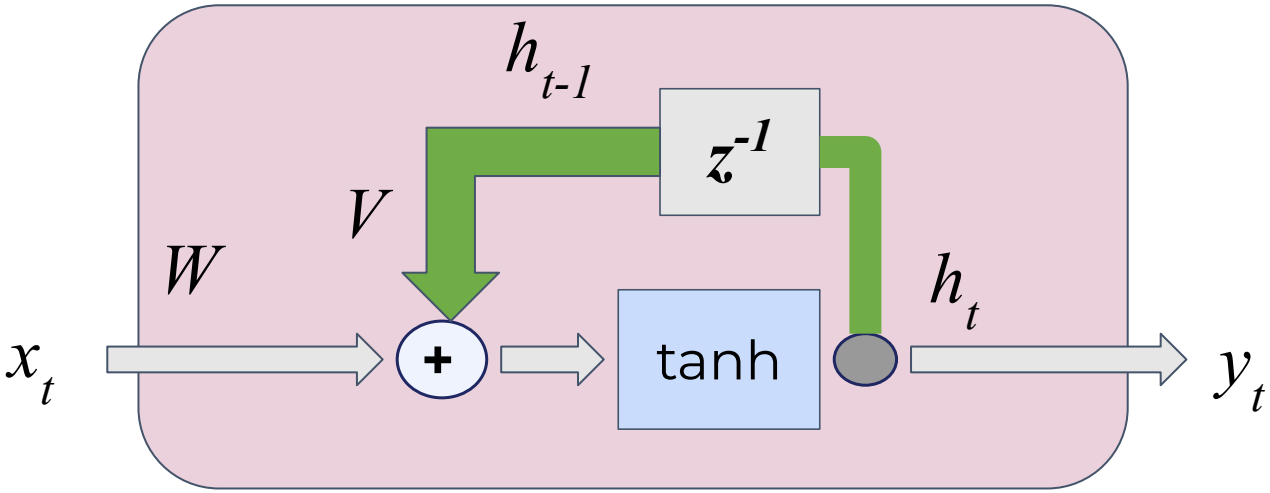

```
class CharsDataset(data.Dataset):
    def __init__(self, prev_chars=3):
        self prev_chars = prev_chars

    with open(path, 'r', encoding='utf-8') as f:
        self.text = f.read()
        self.text = self.text.replace('\ufeff', )
        self.text = re.sub(r'[^A-яA-z0-9.,?;: ]', '', self.text)

        self.text = self.text.lower()
        self.alphabet = set(self.text)
        self.int_to_alpha = dict(enumerate(sorted(self.alphabet)))
        self.num_characters = len(self.alphabet)
        self.onehots = torch.eye(self.num_characters)

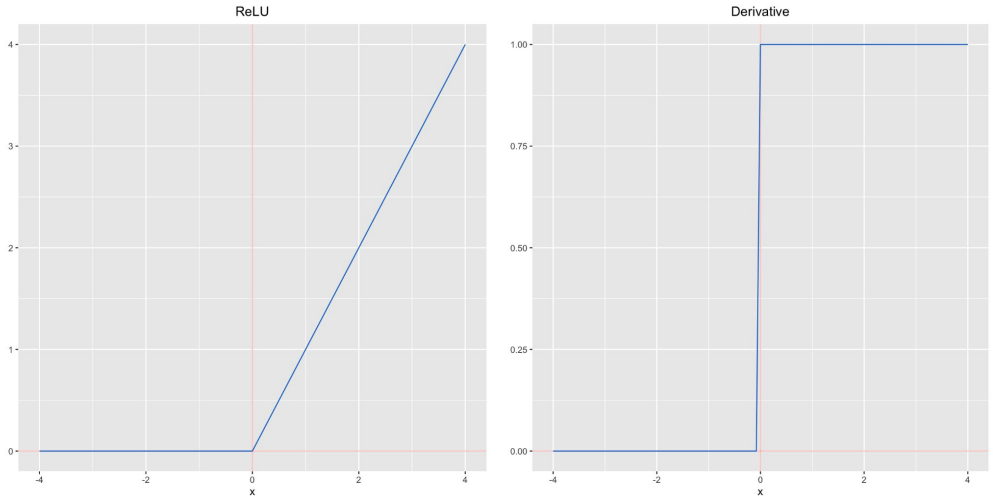
    def __getitem__(self, item):
        _data = torch.vstack([self.onehots[self.alpha_to_int[self.text[x]]]
                              for x in range(item, item+self.prev_chars)])
        ch = self.text[item+self.prev_chars]
        t = self.alpha_to_int[ch]
        return _data, t
```

Исчезающие и взрывающиеся градиенты

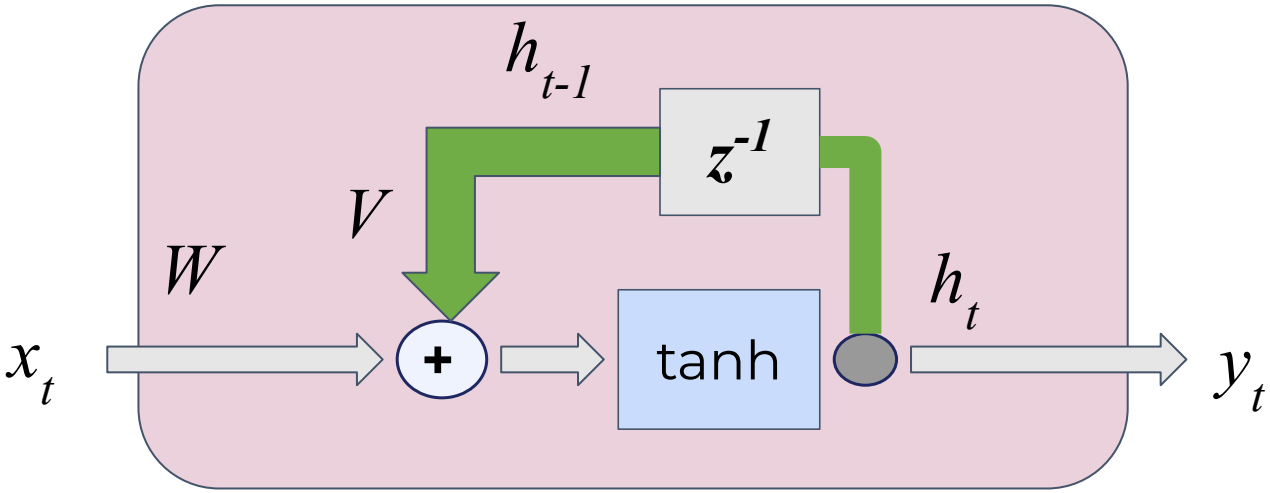


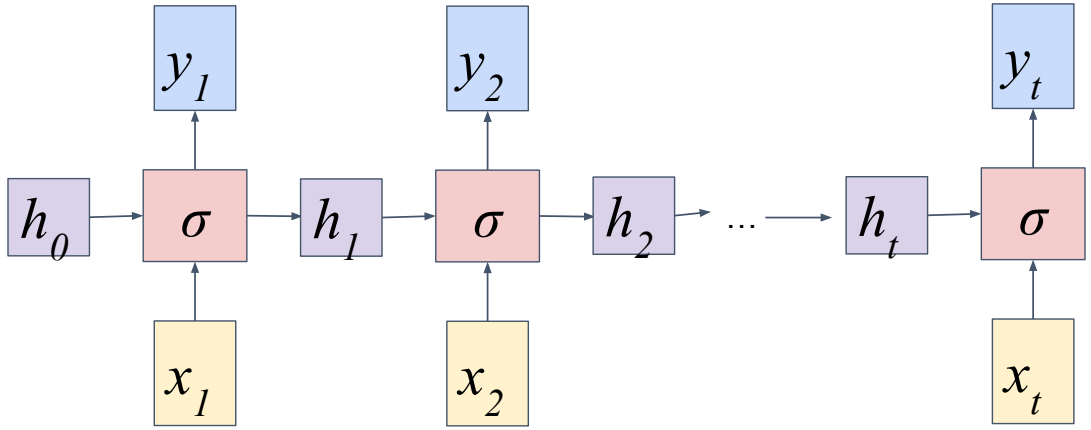
$$h_t = tanh(W x_t + V h_{t-1} + b_x + b_n)$$

$$y = h_t$$



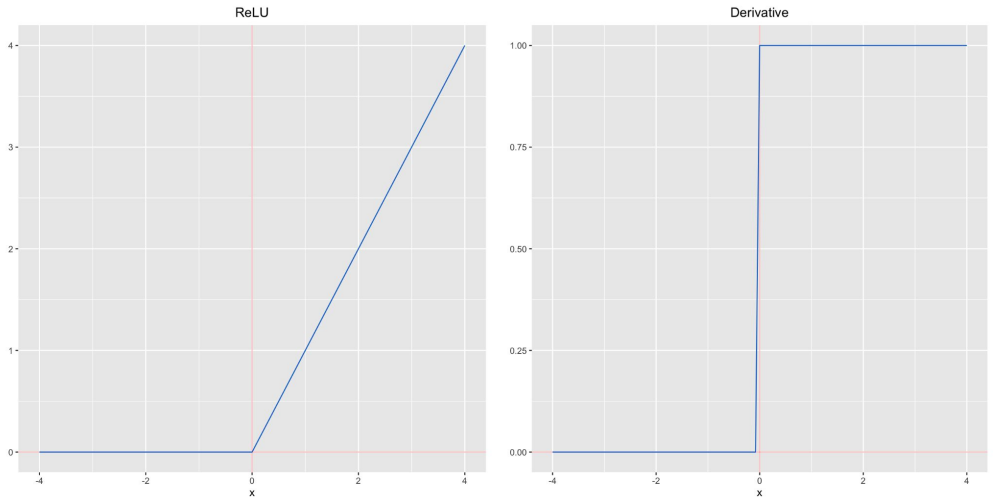
Исчезающие и взрывающиеся градиенты





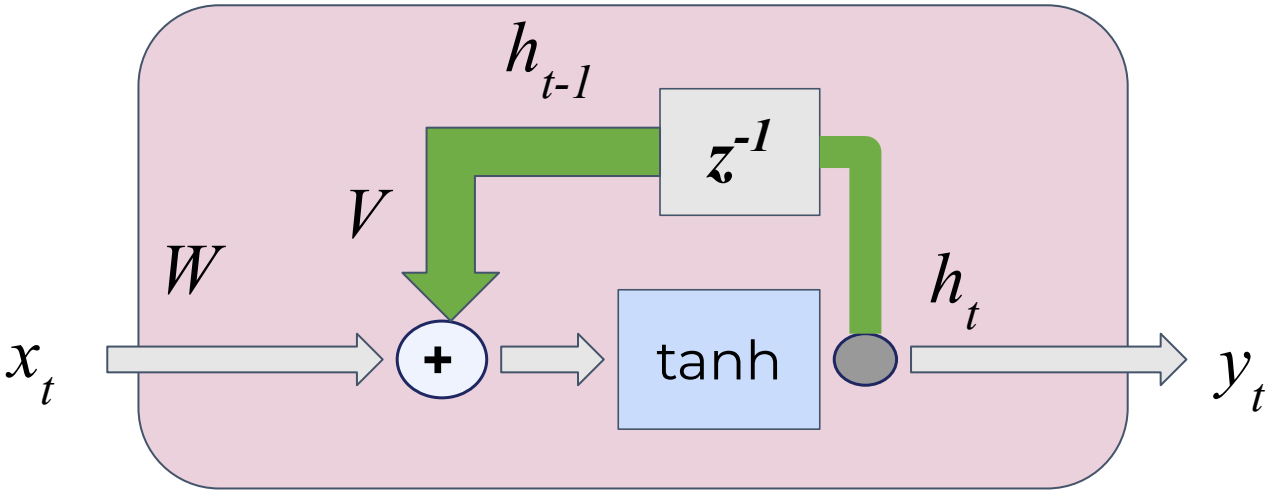
$$h_t = tanh(W x_t + V h_{t-1} + b_x + b_n)$$

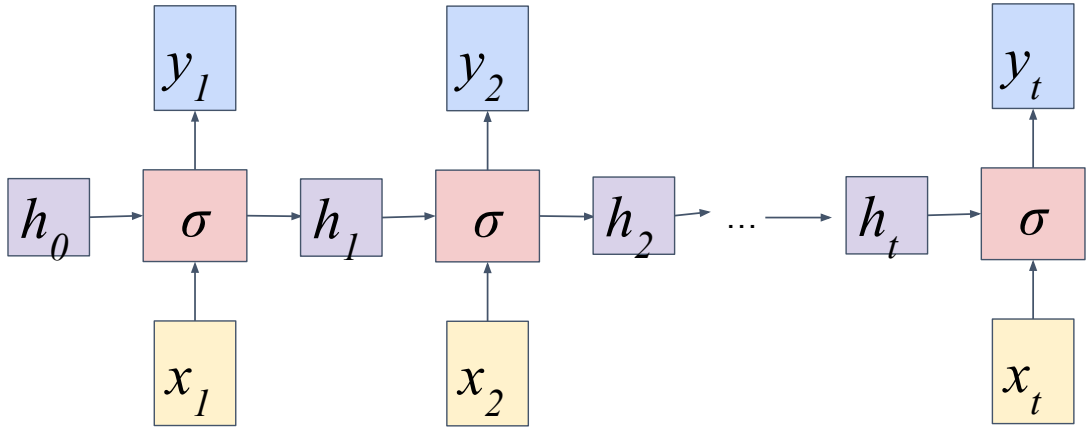
$$y = h_t$$



Исчезающие и взрывающиеся градиенты

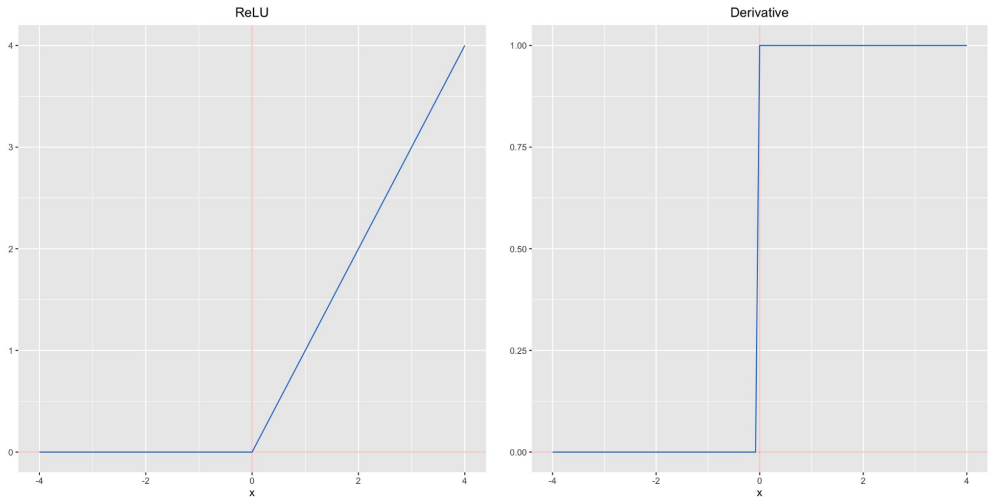
torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)





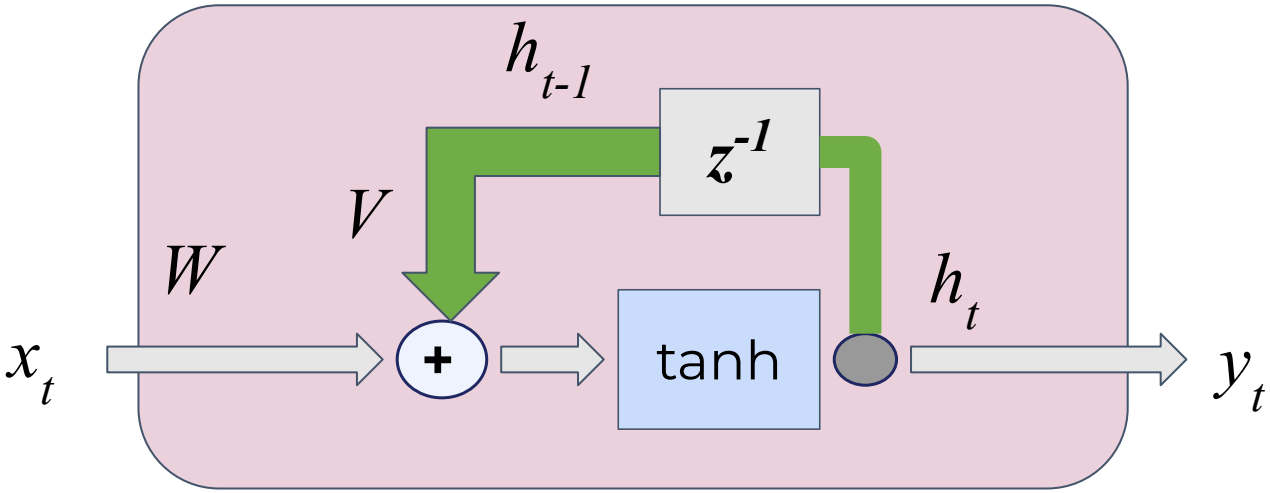
$$h_t = ReLU(W x_t + V h_{t-1} + b_x + b_n)$$

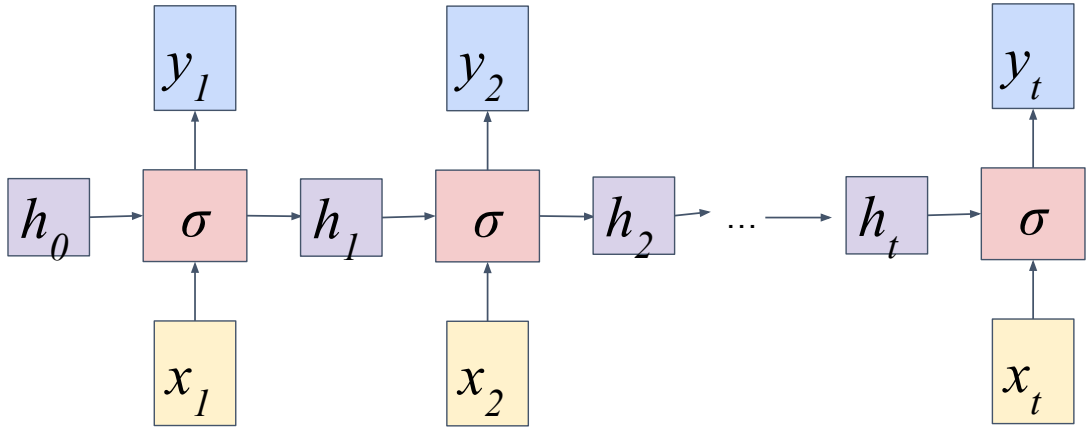
$$y = h_t$$



Исчезающие и взрывающиеся градиенты

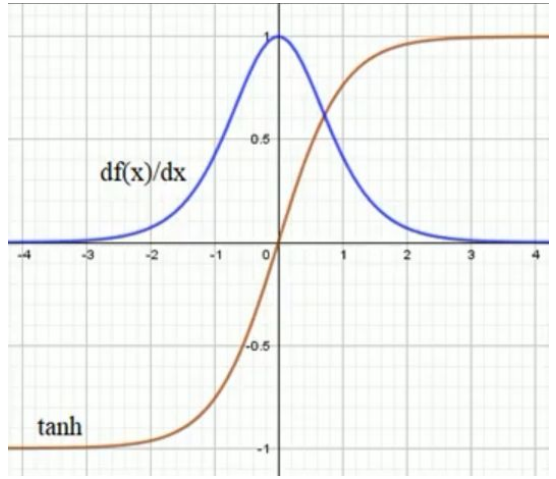
torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)





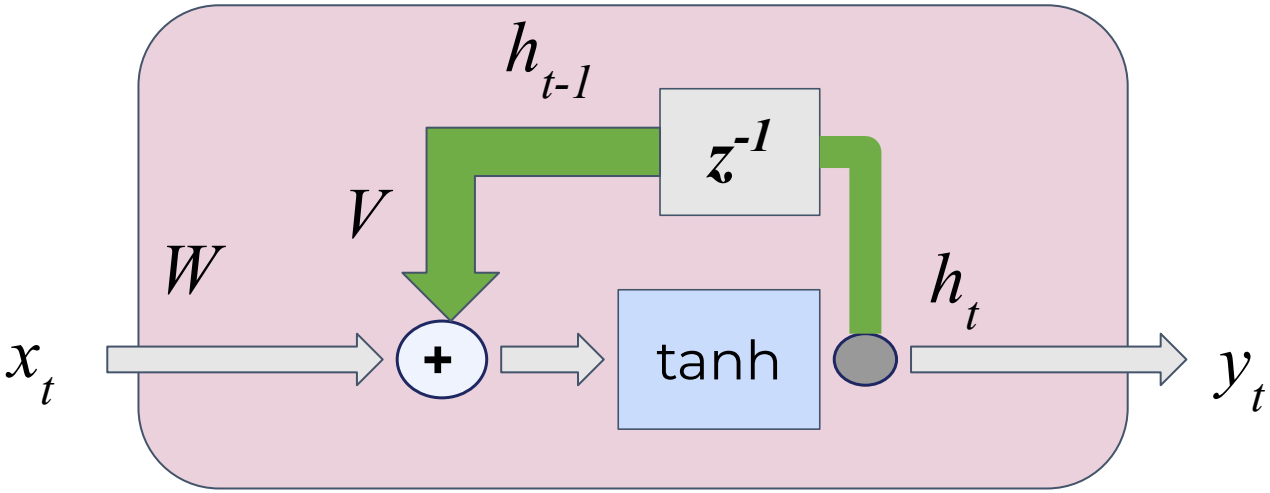
$$h_t = tanh(W x_t + V h_{t-1} + b_x + b_n)$$

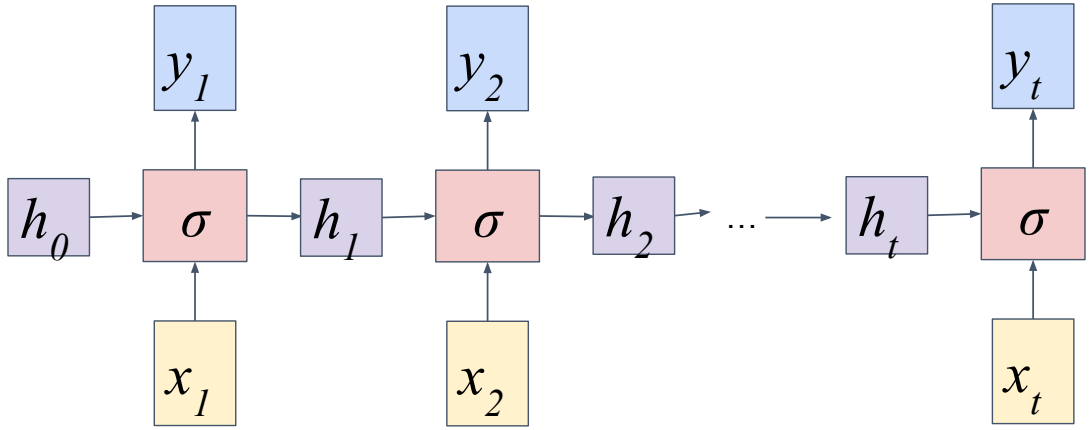
$$y = h_t$$



Исчезающие и взрывающиеся градиенты

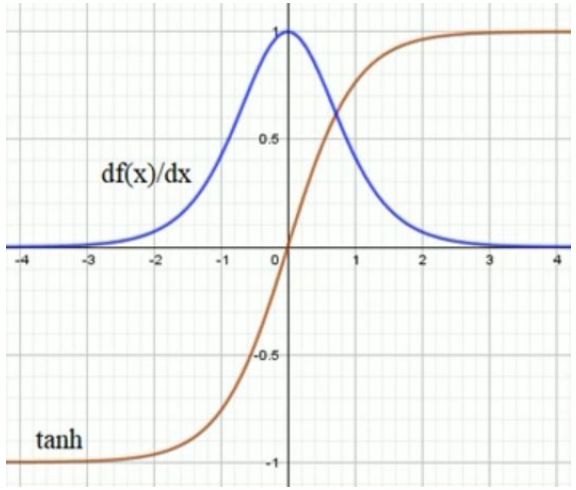
torch.nn.RNN(input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False)





$$h_t = sigma(W x_t + V h_{t-1} + b_x + b_n)$$

$$y = h_t$$



Задачи с известной последовательностью

Продолжайте учиться. Трудности в конце концов создают сильных людей.

Пацаны ваще ребята, внатуре четко… Умеете, могете!

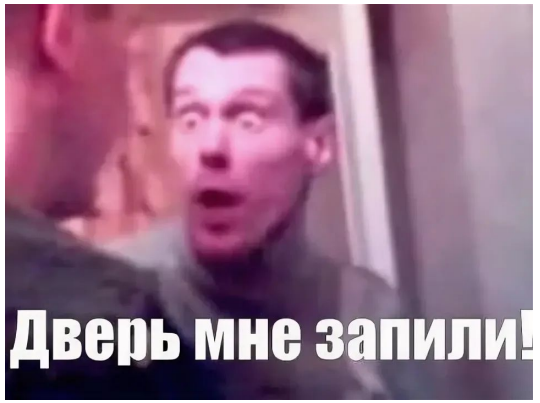




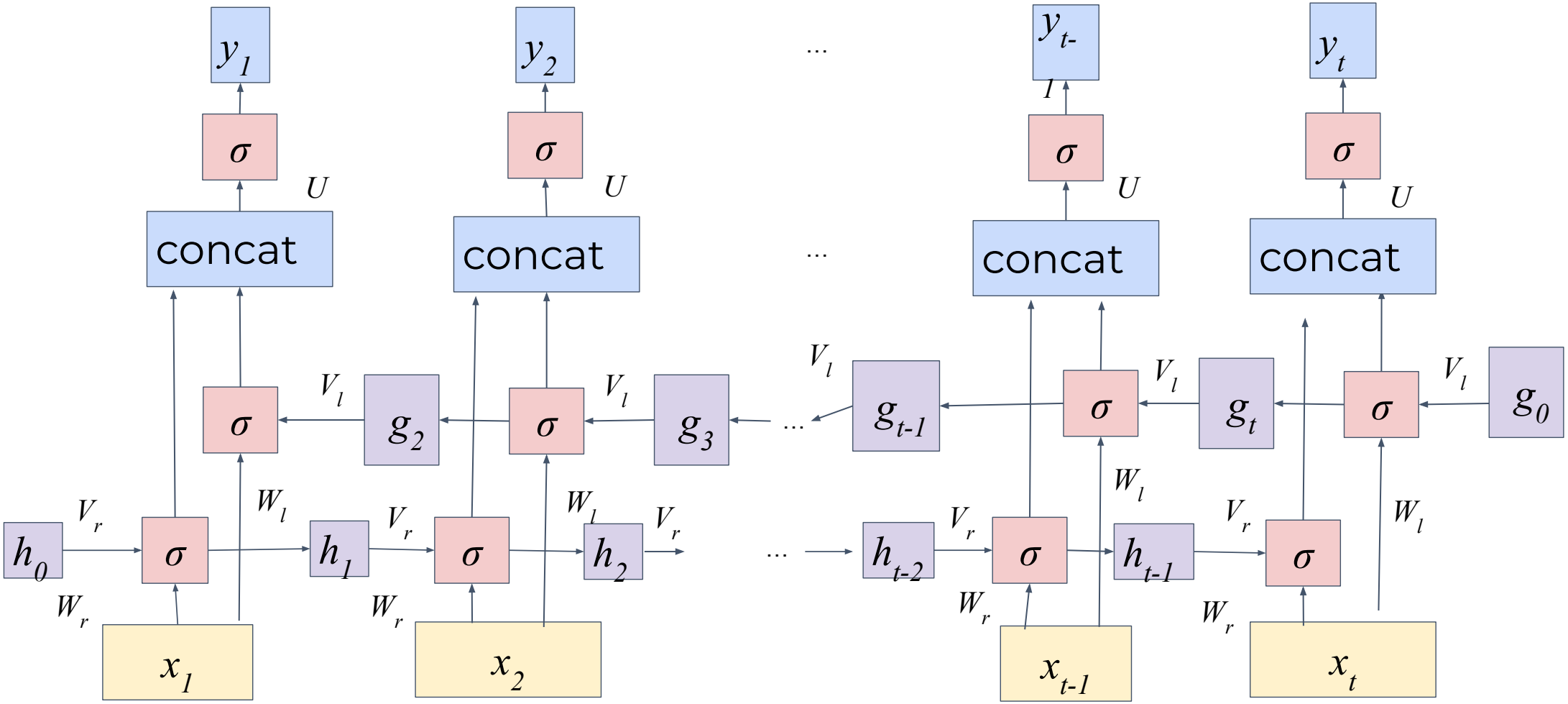
Мне публично выступить? Что вы?! Это точно не мое!

Чего тебе надо у меня дома?





Двунаправленные (bidirectional) RNN



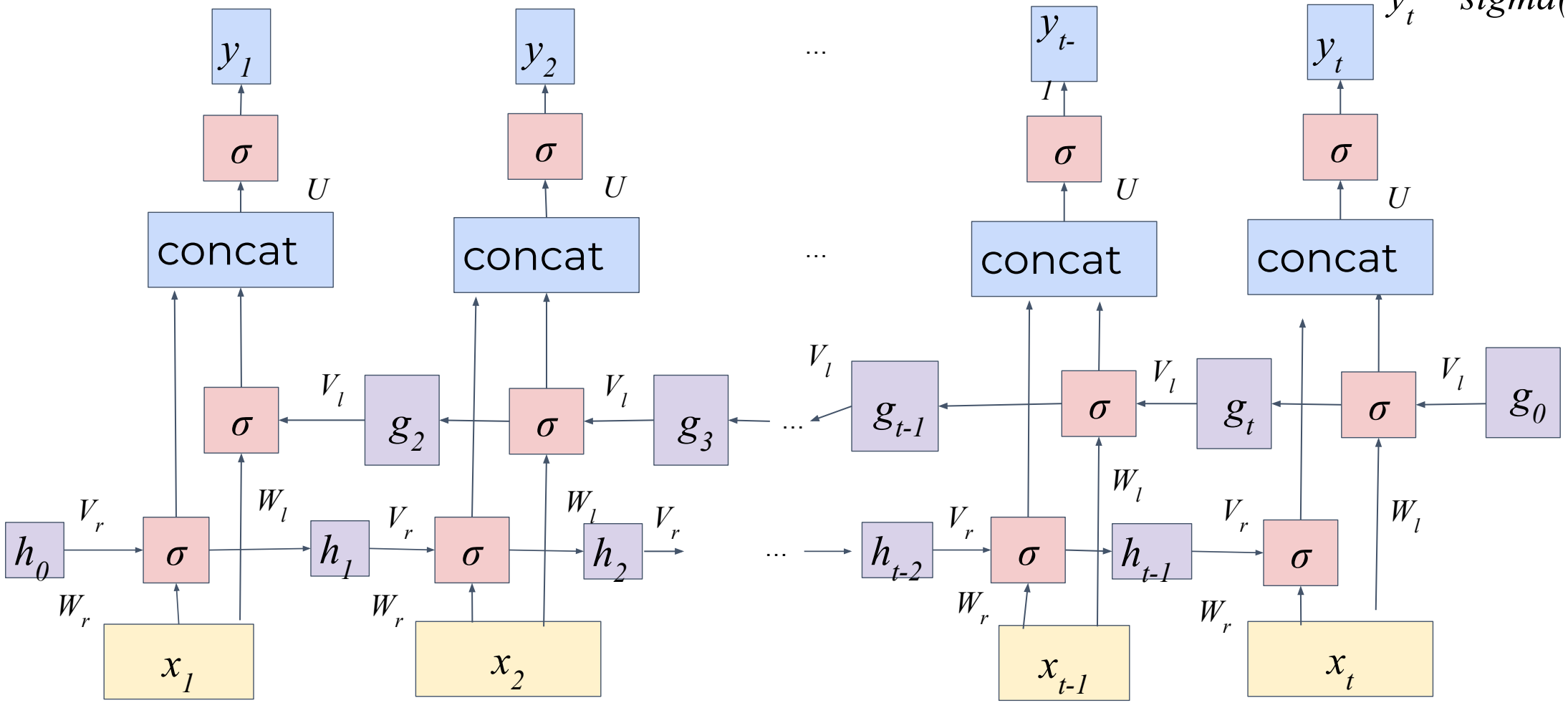
Двунаправленные (bidirectional) RNN

$$h_t = sigma(W_r x_t + V_r h_{t-1} + b_x + b_n)$$

$$g_t = sigma(W_l x_l + V_l g_{t-1} + b_x^l + b_x^l)$$

$$o_t = [h_t, g_t]$$

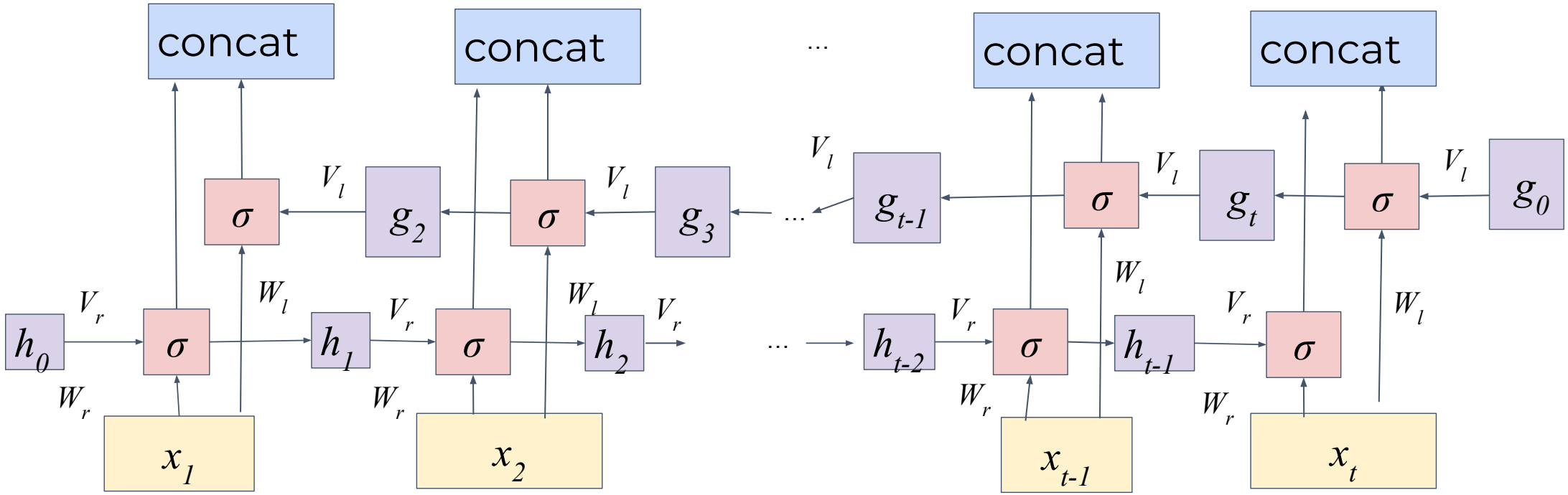
$$y_t = sigma(U * o_t + b_y)$$



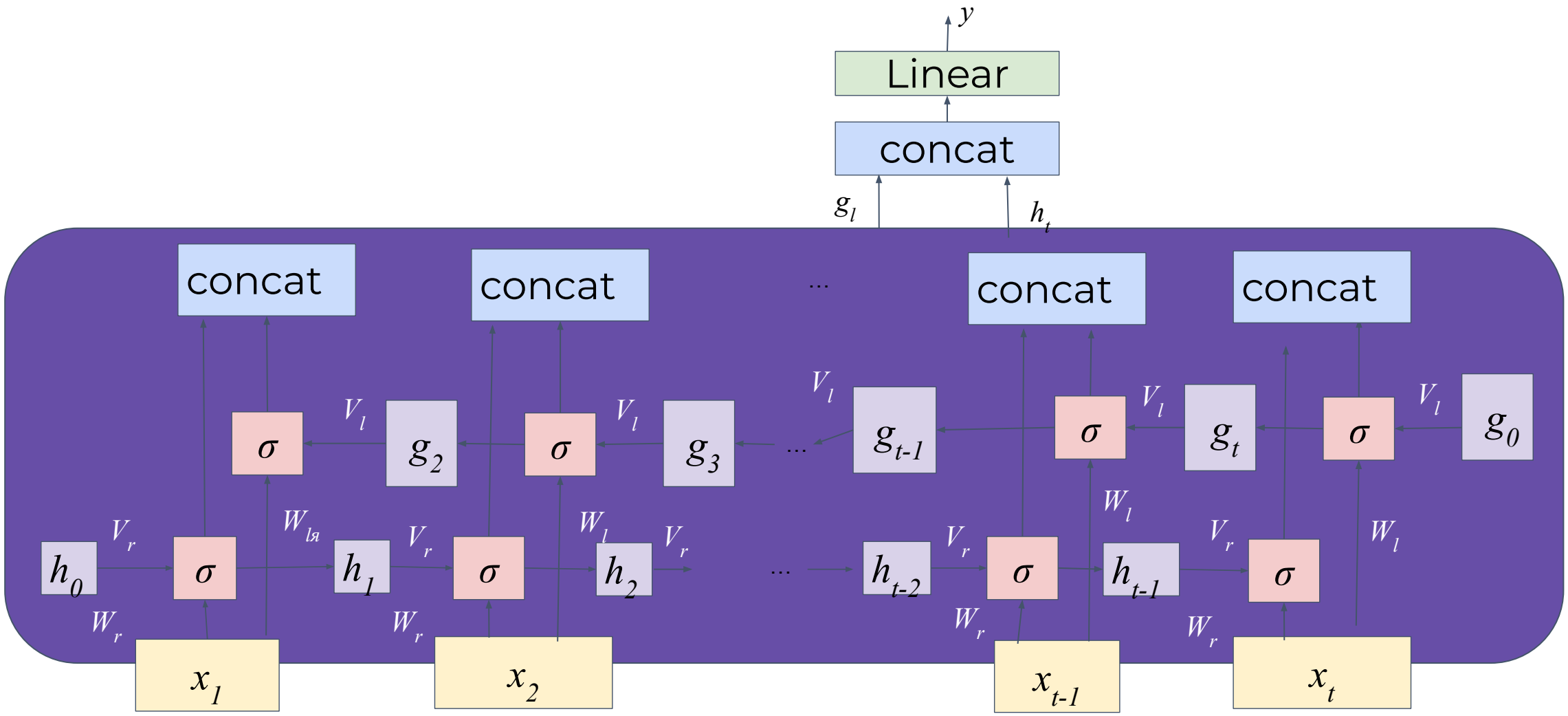
Двунаправленные (bidirectional) RNN

torch.nn.**RNN**(input_size, hidden_size, batch_first=True, **bidirectional=True**, ...)

```
rnn = nn.RNN(input_size, hidden_size, batch_first=True, bidirectional=True)
y, h = rnn(x, h0)
```



Модель RNN-сети для сентимент-анализа

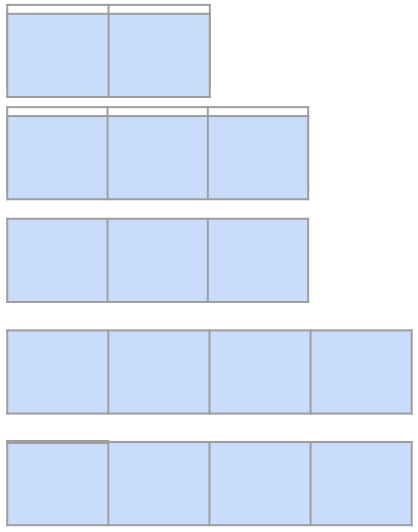


Формат обучающей выборки

seq_lenght

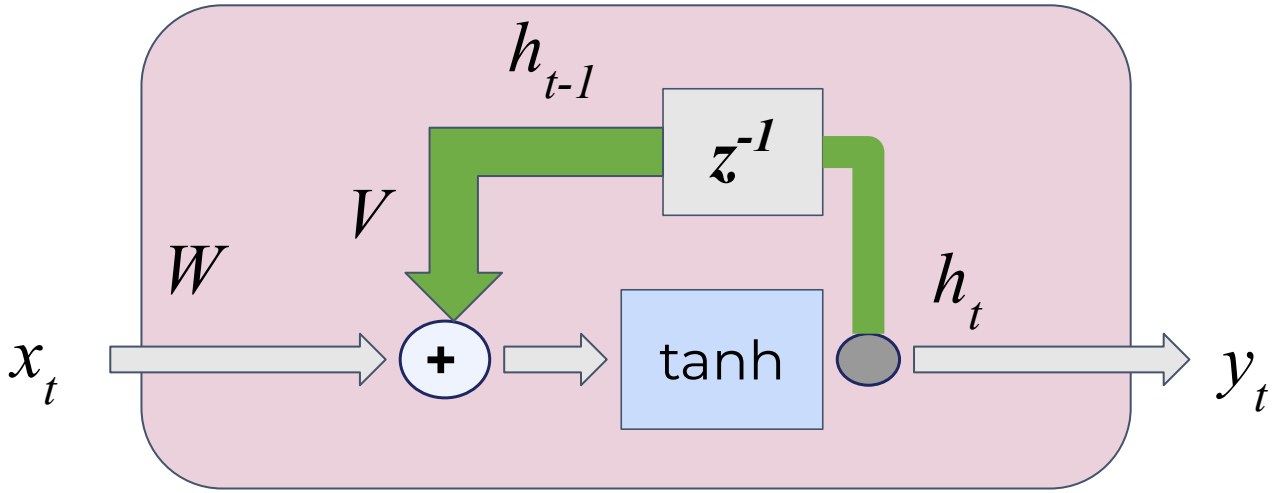
batch_Size

Мне	нравится	работа			
Это	шедевр	изобразительного	искусства		
Жизнь	слишком	коротка	чтобы	быть	несчастной



,,,

Простой RNN-слой



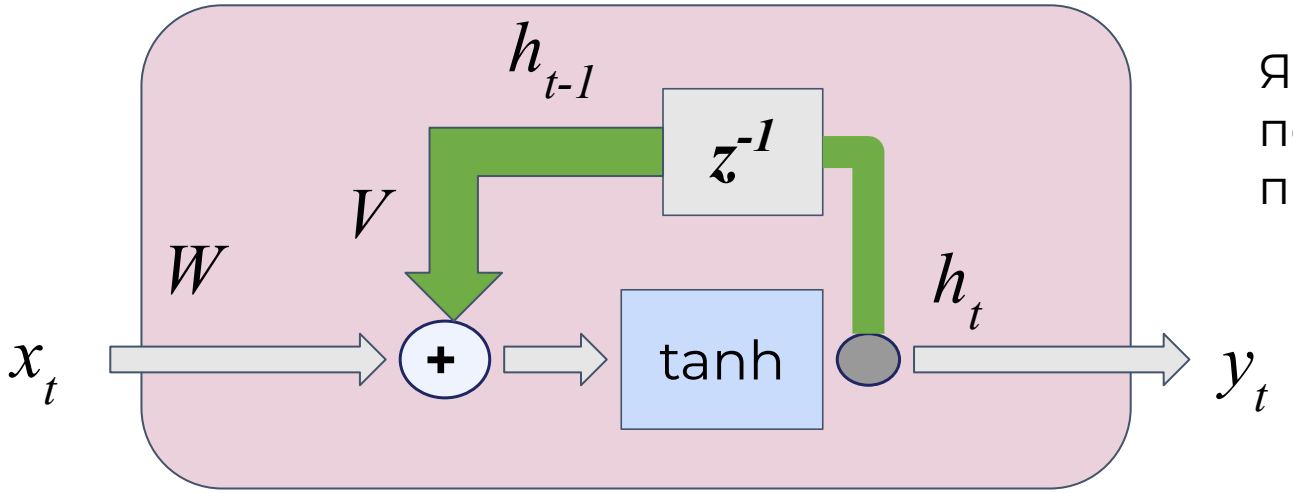
Я очень люблю *программировать*, …, поэтому в будущем хочу стать программистом.

$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

$$y = h_t$$

Простой RNN-слой



Я очень люблю *программировать*, ..., поэтому в будущем хочу стать программистом.

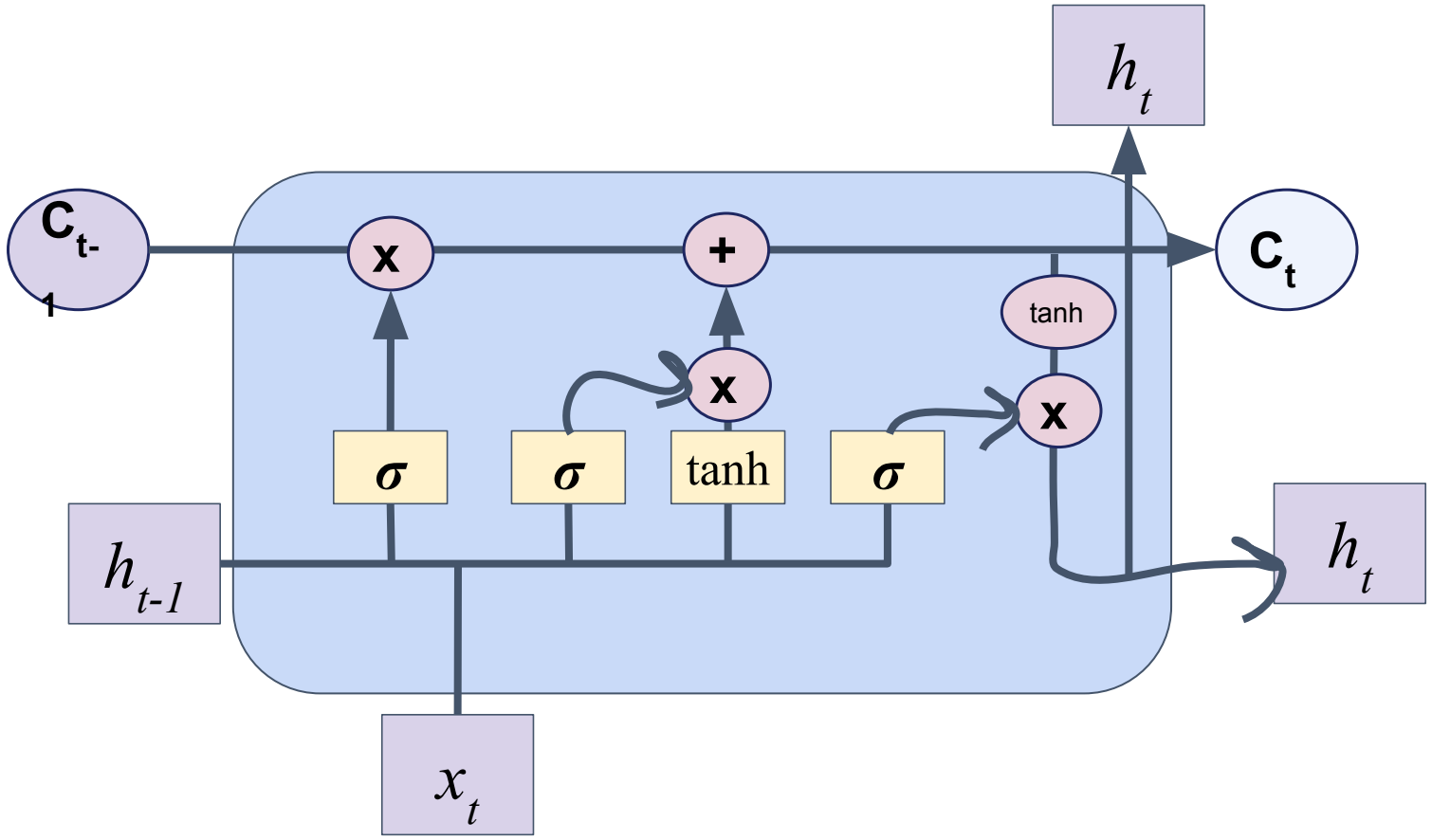
$$a = W x_t + V h_{t-1} + b_x + b_n$$

$$h_t = tanh(a_t)$$

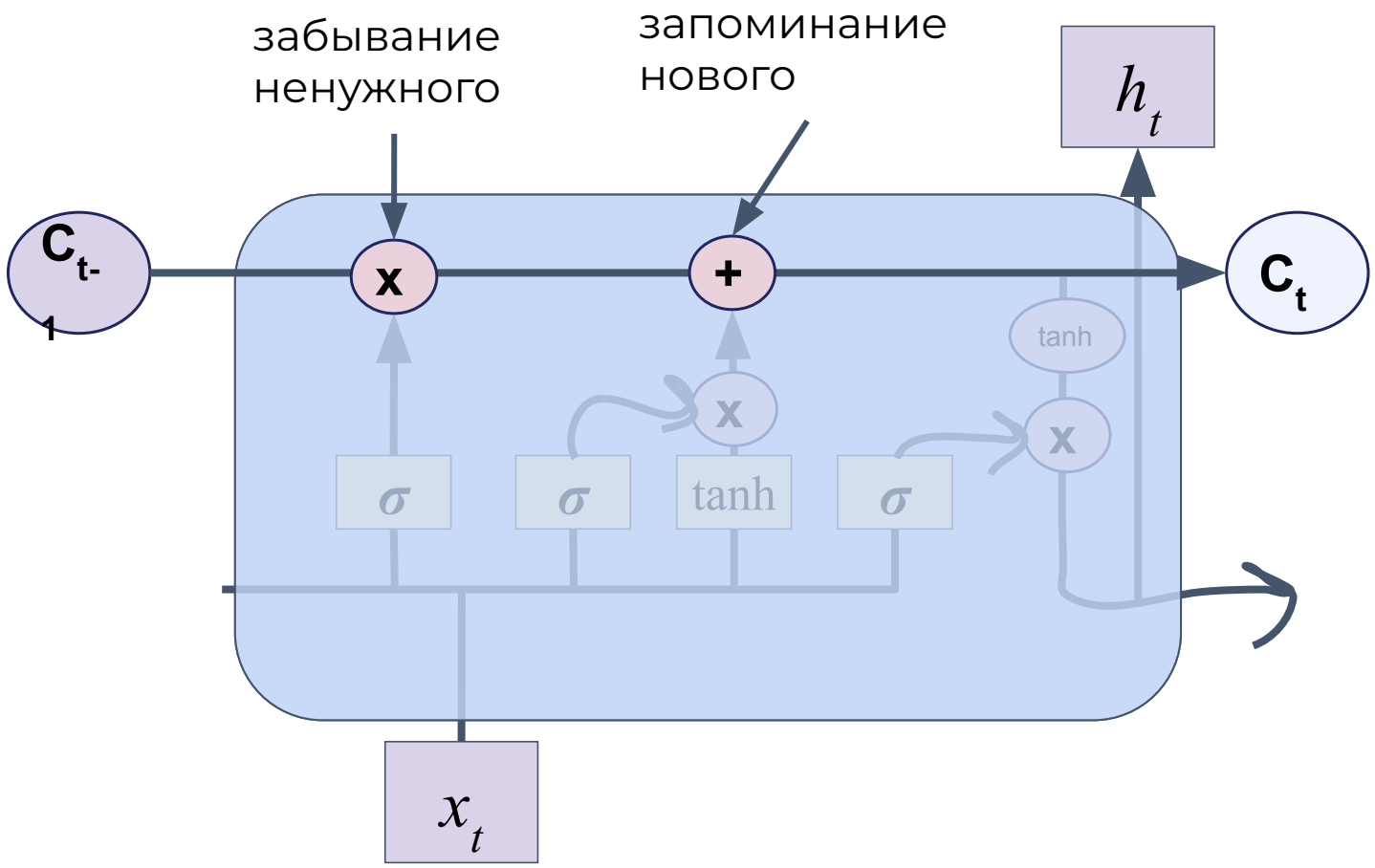
$$y = h_t$$

LSTM (Long Short-Term Memory) - долга краткосрочная память

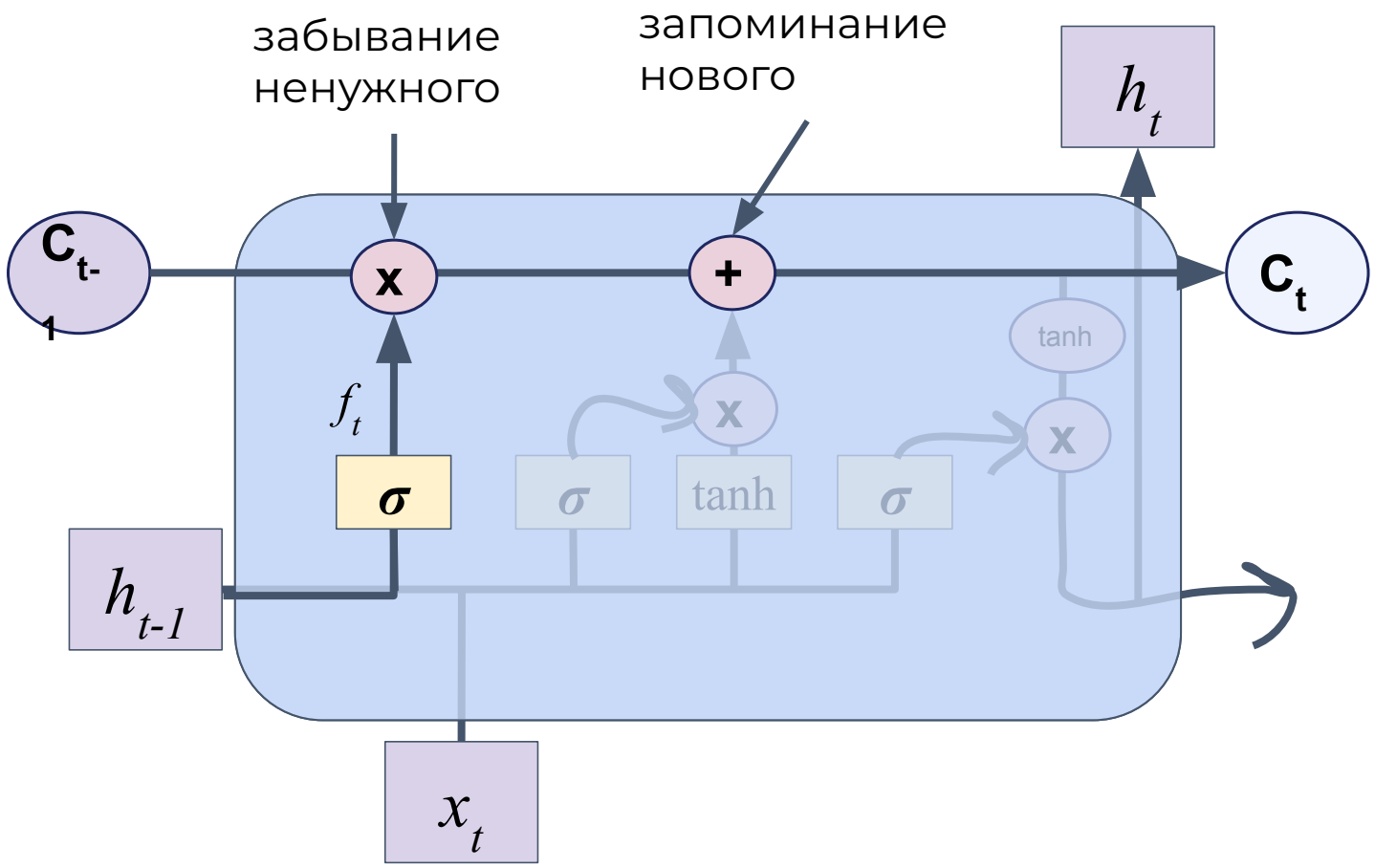
Архитектура LSTM



Архитектура LSTM



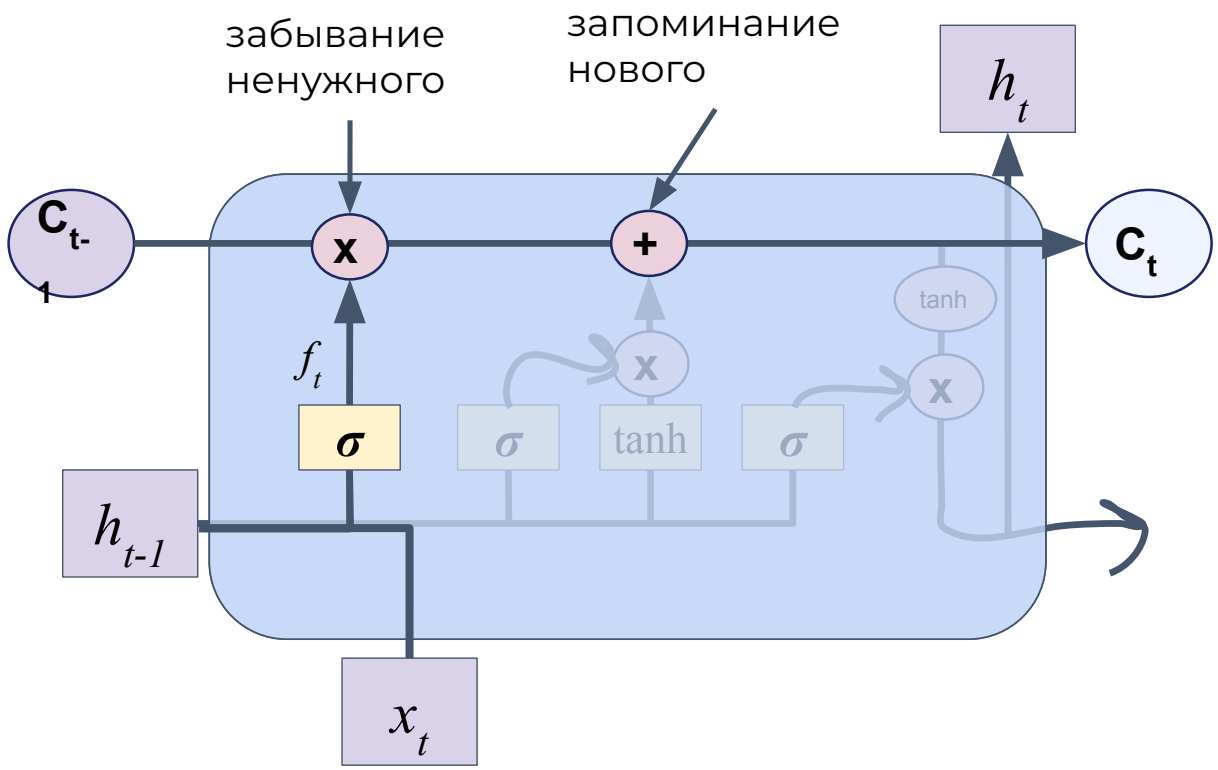
Архитектура LSTM



$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

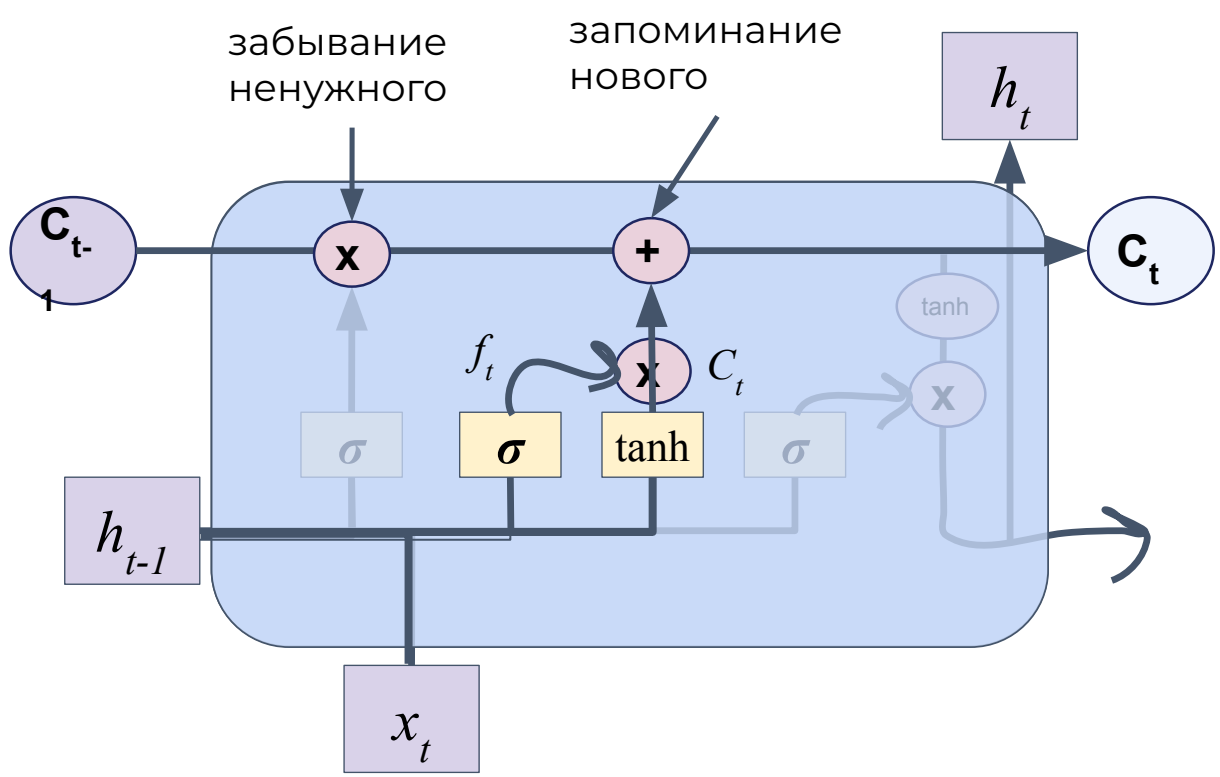
$$f_t = [0,2;\ 0,3;\ 0,9;\ 0,1;\ 0,5,\ldots]^T$$

Архитектура LSTM



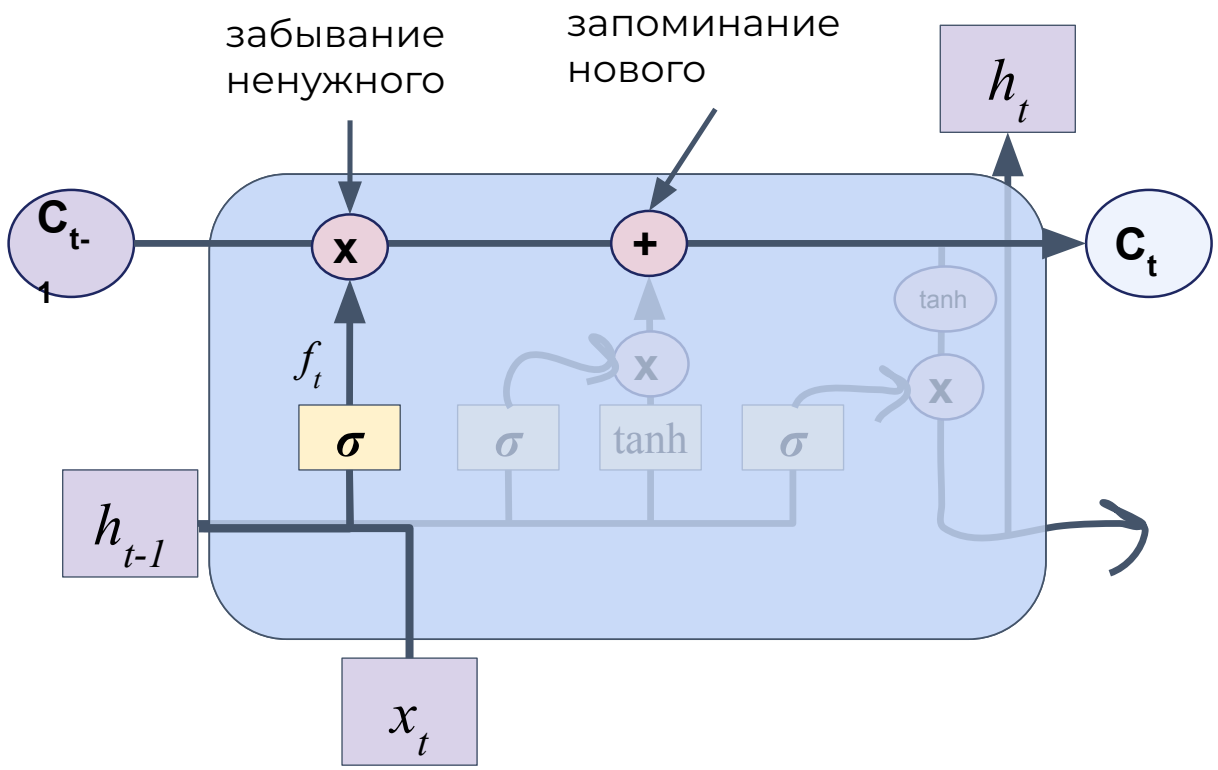
$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

$$f_t = [0,2;\ 0,3;\ 0,9;\ 0,1;\ 0,5,\ldots]^T$$



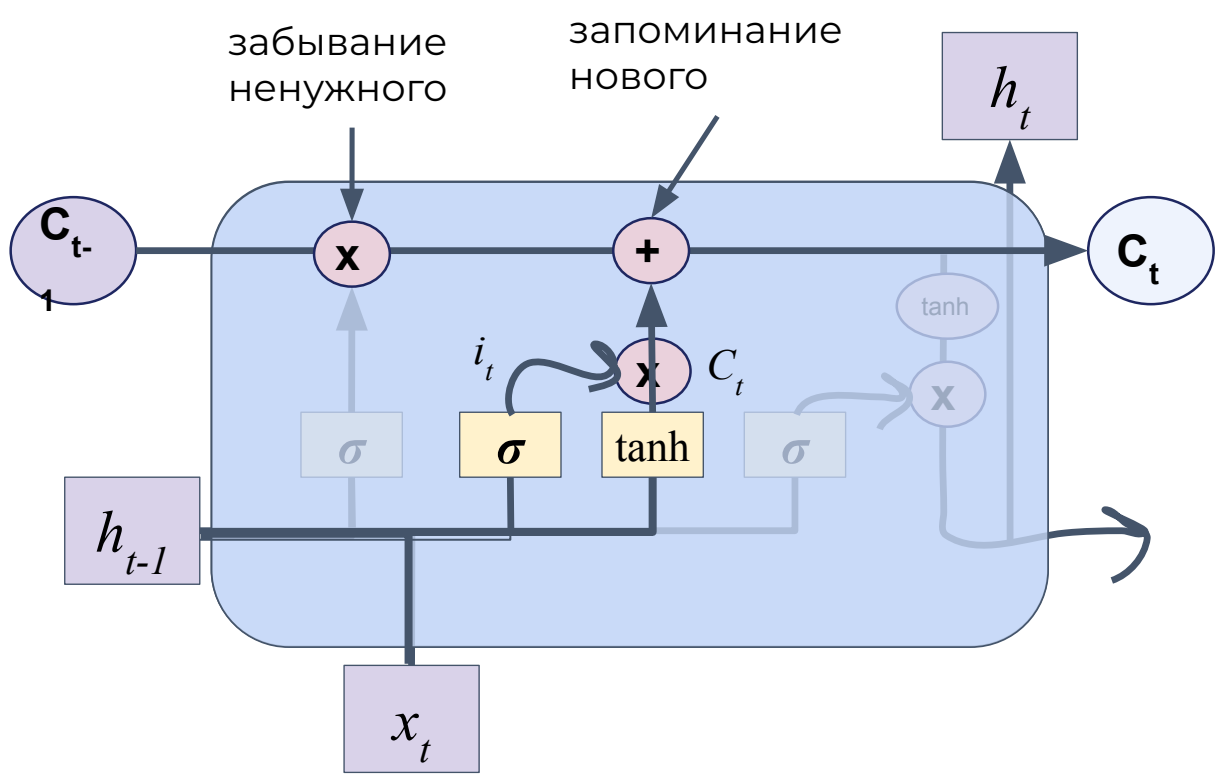
$$i_t = \sigma(W_p [h_{t-1}, x_t] + b_{fi}) \quad C_t = tanh\,(W_C [h_{t-1}, x_t] + b_C)$$

Архитектура LSTM



$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

$$f_t = [0,2;\ 0,3;\ 0,9;\ 0,1;\ 0,5,\ldots]^T$$



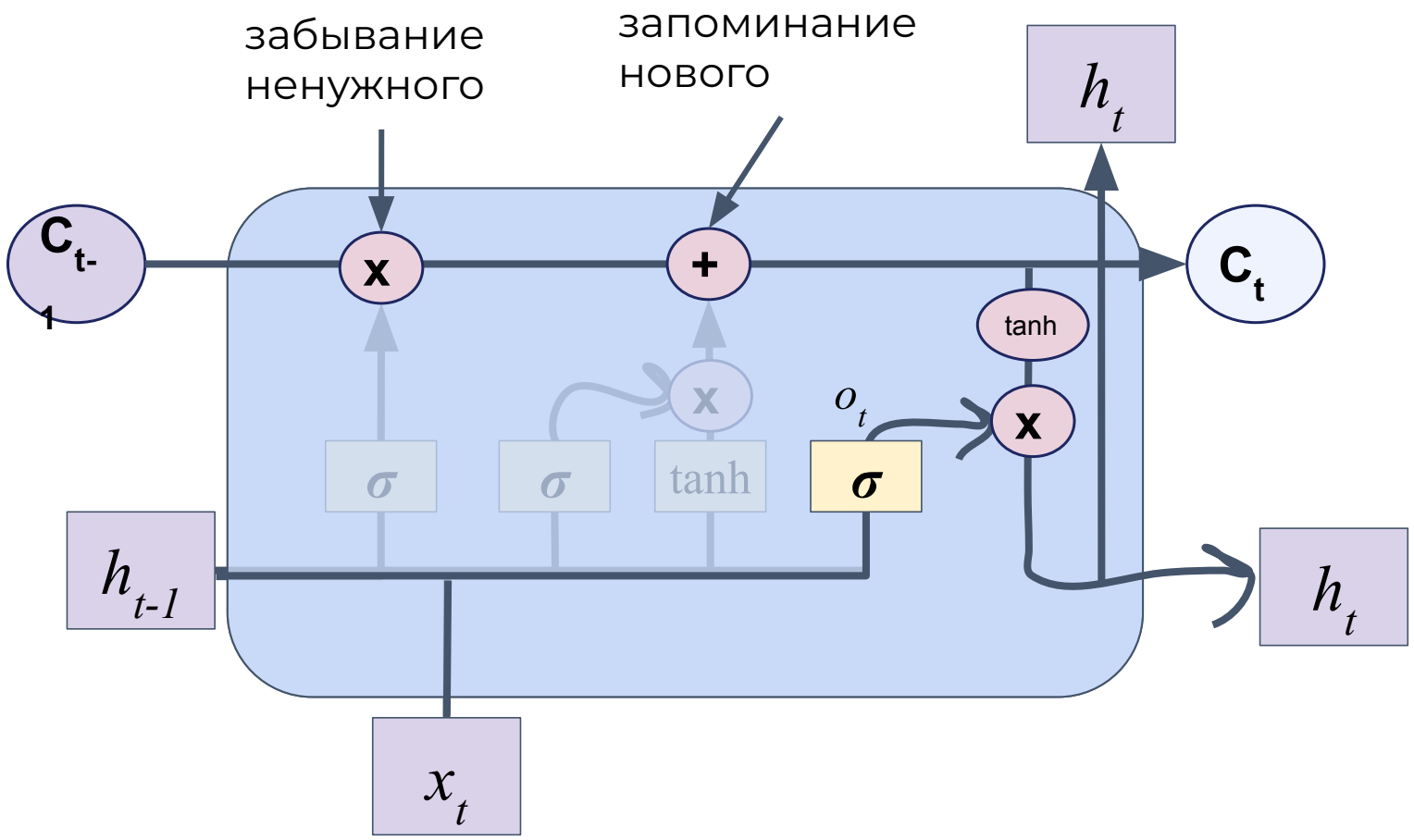
$$i_t = \sigma(W_p [h_{t-1}, x_t] + b_{fi})$$

$$C_t = tanh\,(W_C [h_{t-1}, x_t] + b_C)$$

$$a_t = i_t \otimes C_t$$

$$C_t = f_t \otimes C_{t-1} + a_t$$

Архитектура LSTM



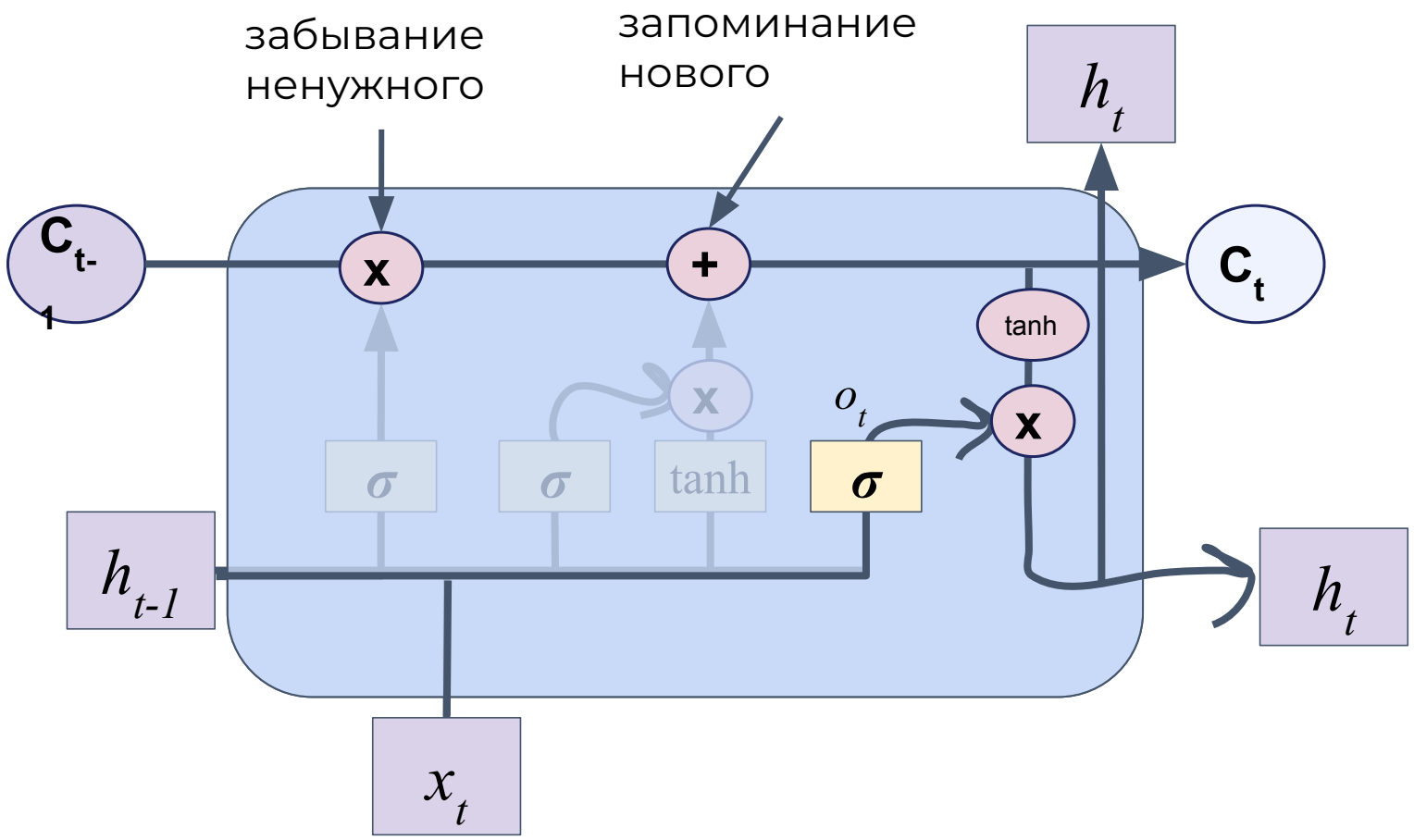
$$i_t = \sigma(W_o\,[h_{t-1}, x_t] + b_o)$$

$$a_t = o_t \otimes \tanh(C_t)$$

$$f_t = \sigma(W_f\,[h_{t-1}, x_t] + b_f)$$

$$f_t = [0,2;\ 0,3;\ 0,9;\ 0,1;\ 0,5,\ldots]^T$$

Архитектура LSTM



$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_p [h_{t-1}, x_t] + b_t)$$

$$C_t = tanh\,(W_C [h_{t-1}, x_t] + b_C)$$

$$a_t = i_t \otimes C_t$$

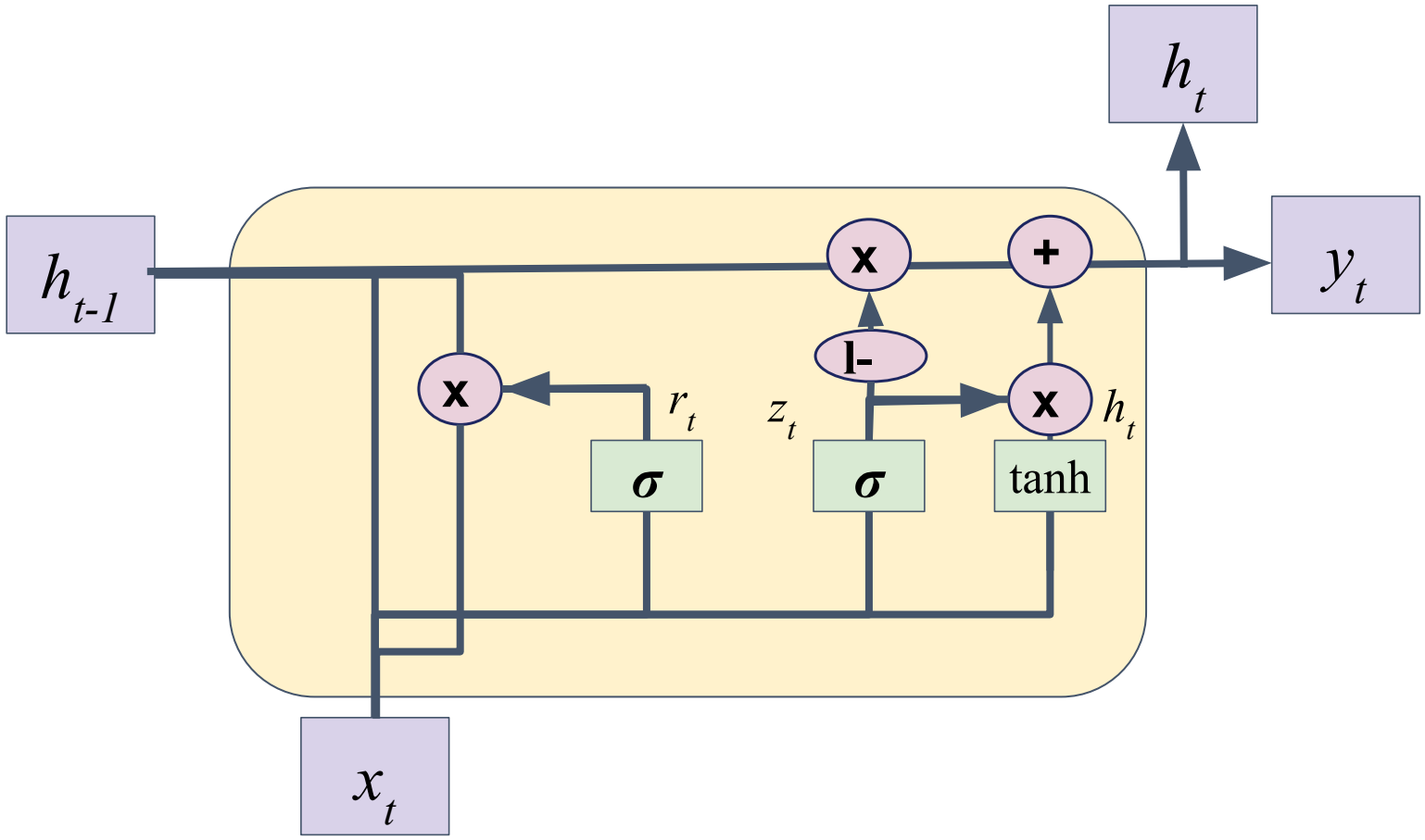
$$C_t = f_t \otimes C_{t-1} + a_t$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

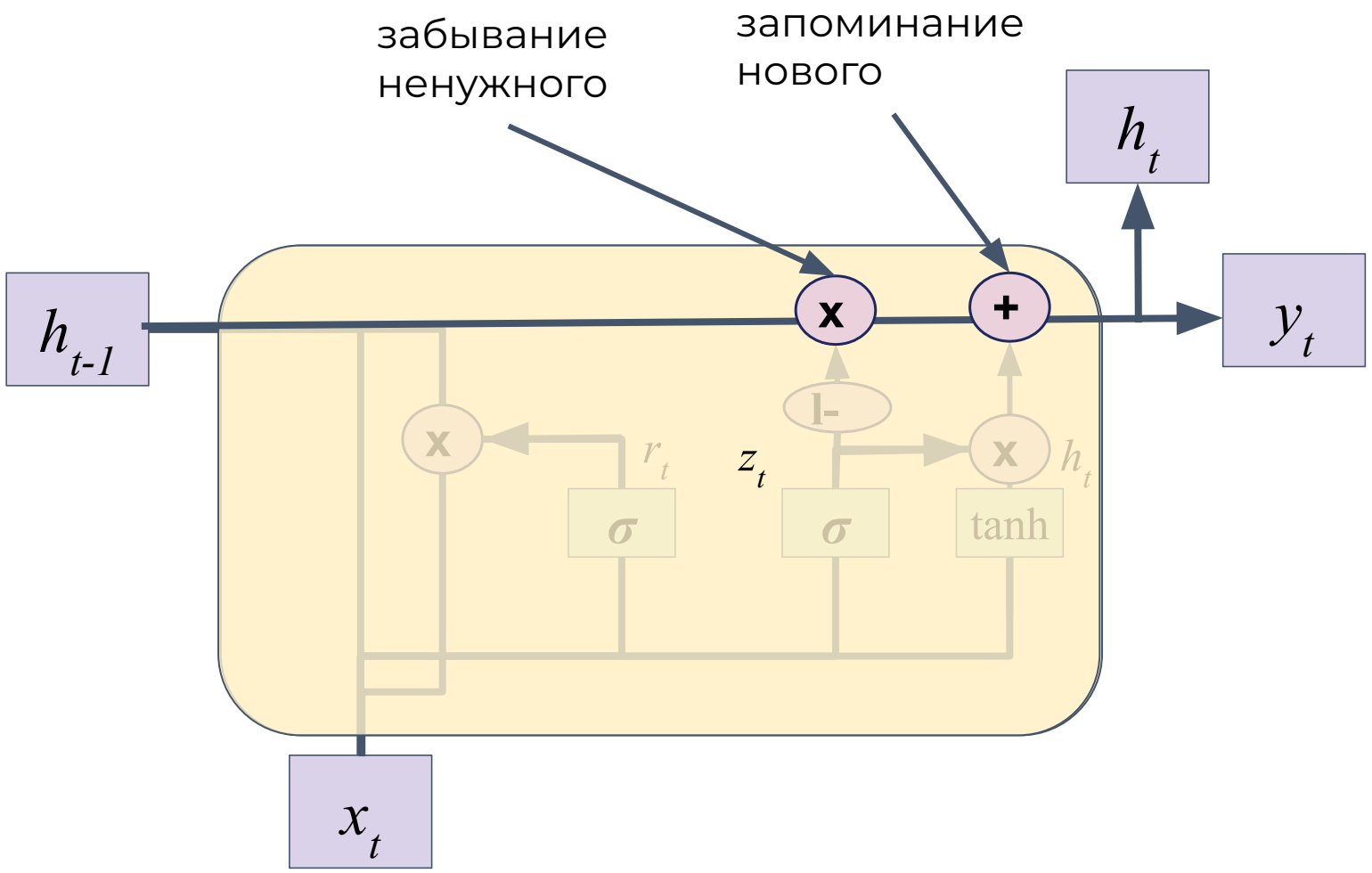
$$h_t = o_t \otimes \tanh(C_t)$$

Архитектура GRU

Gated Recurrent Units (GRU)

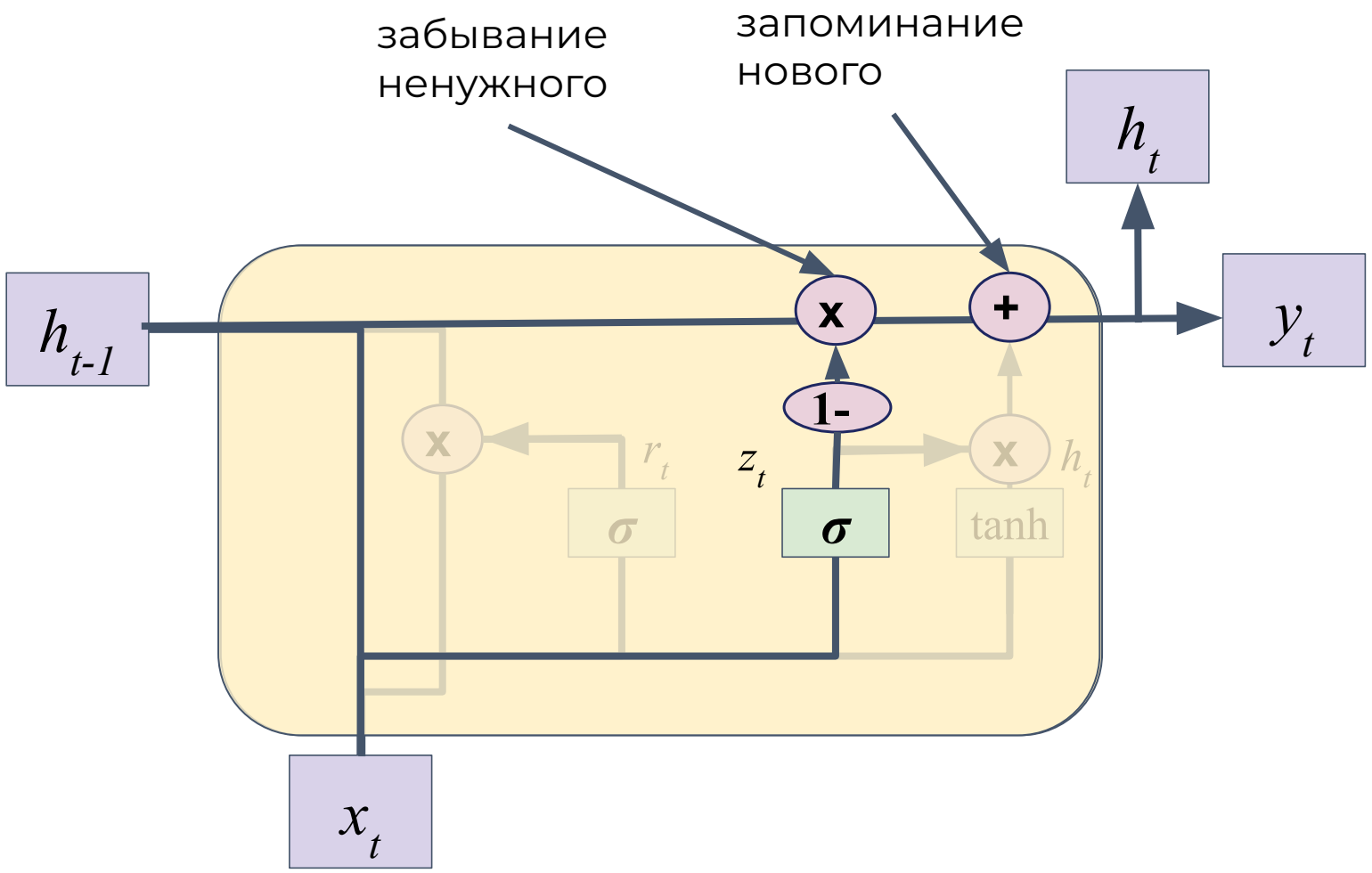


Архитектура GRU



Gated Recurrent Units (GRU)

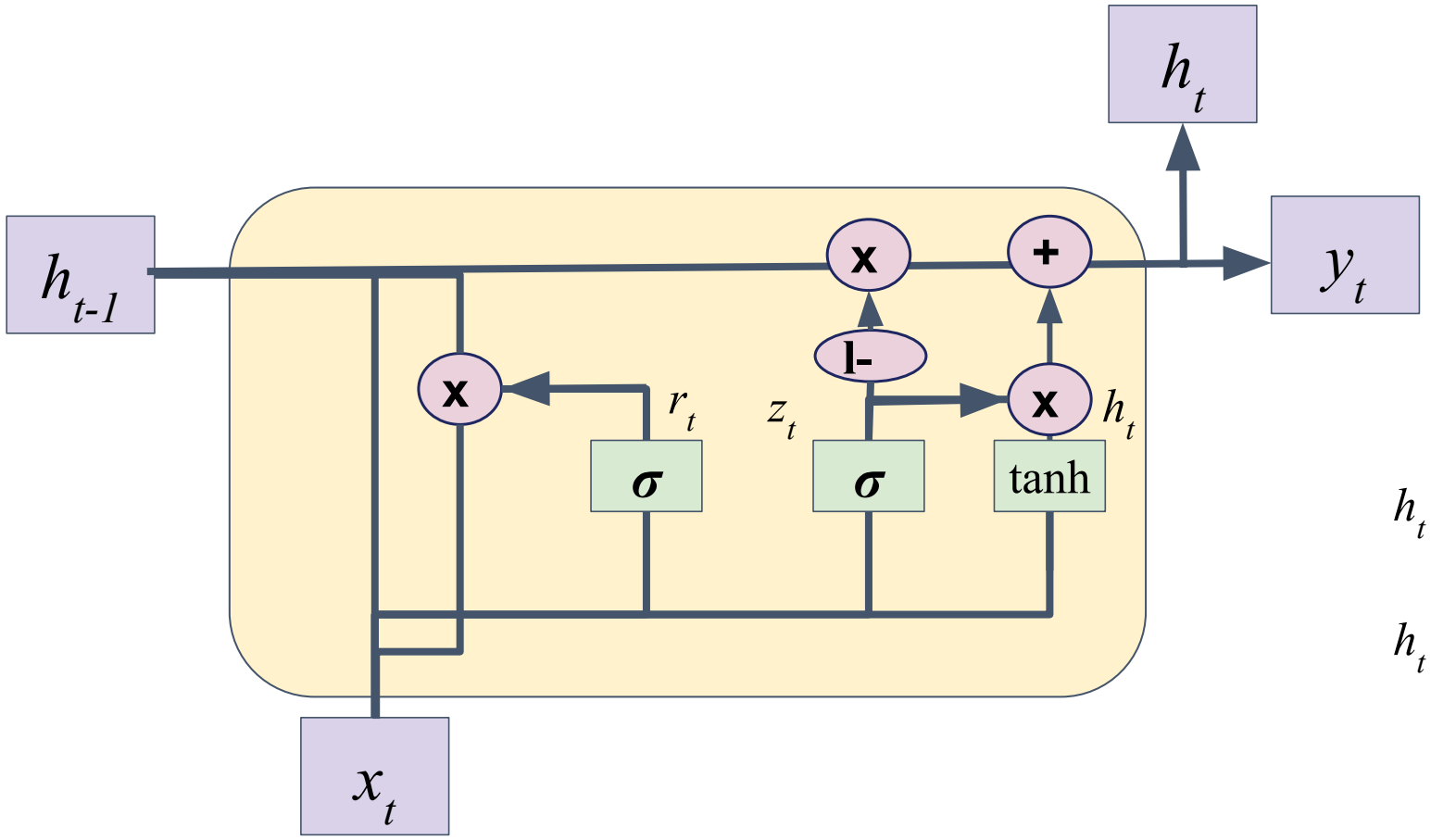
Архитектура GRU



$$z_t = \sigma(W_z [h_{t-1}, x_t] + b_z)$$

$$f_t = h_{t=1} \otimes (1 - z_t)$$

Архитектура GRU



Gated Recurrent Units (GRU)

$$z_t = \sigma(W_z [h_{t-1}, x_t] + b_z)$$

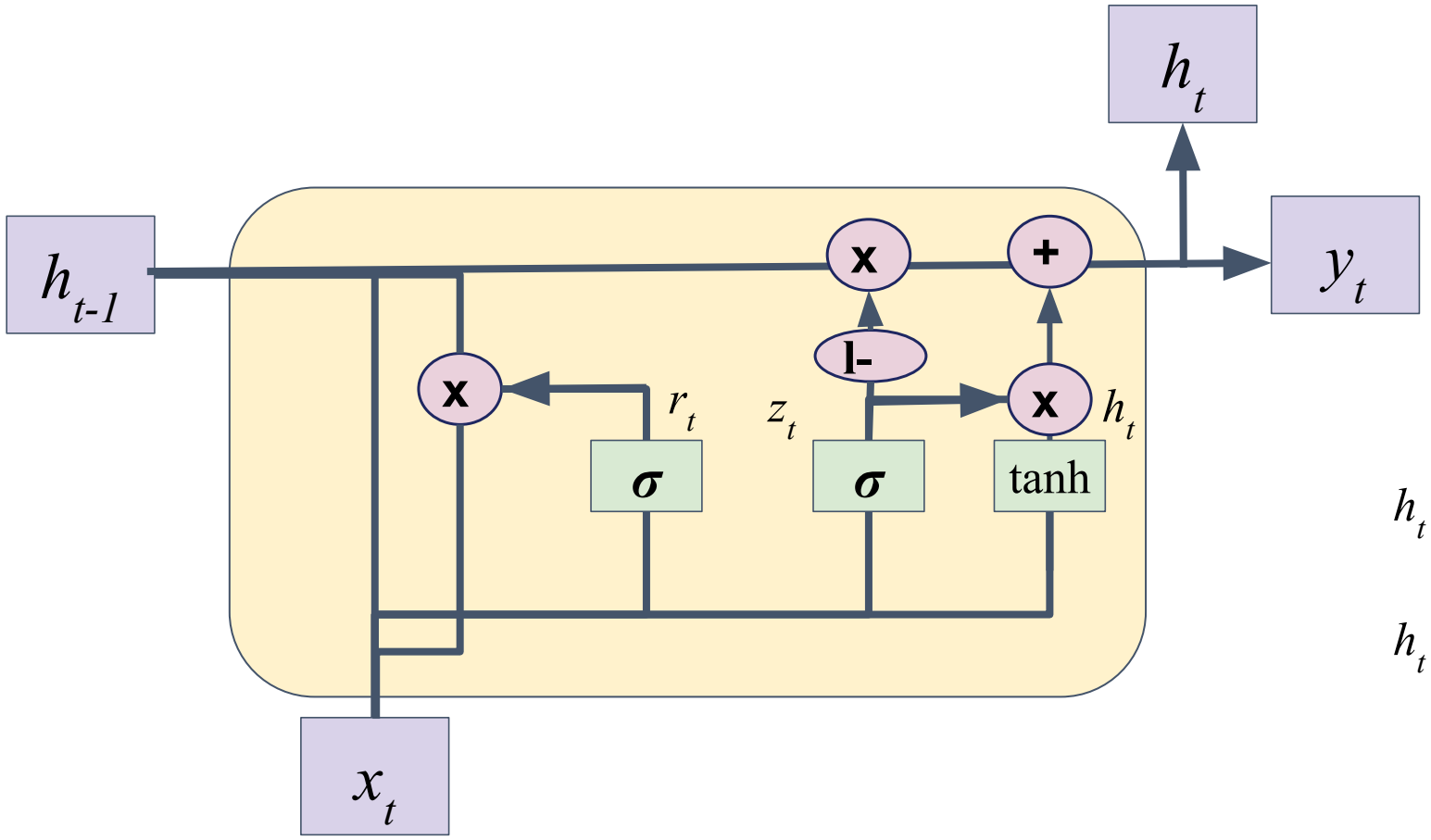
$$f_t = h_{t=1} \otimes (1 - z_t)$$

$$r_t = \sigma(W_p [h_{t-1}, x_t] + b_r)$$

$$h_t = tanh\ (W_{xh}\, x_t + W_{hh}\, (r_t \otimes h_{t-1}) + b_h)$$

$$h_t = h_{t=1} \otimes (1 - z_t) + h_t \otimes z_t$$

Архитектура GRU



Gated Recurrent Units (GRU)

$$z_t = \sigma(W_z\,[h_{t-1}, x_t] + b_z)$$

$$f_t = h_{t=1} \otimes (1 - z_t)$$

$$r_t = \sigma(W_p\,[h_{t-1}, x_t] + b_r)$$

$$h_t = tanh\,(W_{xh}\,x_t + W_{hh}\,(r_t \otimes h_{t-1}) + b_h)$$

$$h_t = h_{t=1} \otimes (1 - z_t) + h_t \otimes z_t$$

torch.nn.GRU(input_size, hidden_size, num_layers=1, bias=True, batch_first=False, dropout=0.0, bidirectional=False, ...)